

CCTBX.XFEL4MFX - Stable

Compiling Github Issues documented by Fred and answers from chats with Aaron, Dan, David, and other cctbx.xfel developers. Adapted Pam's notes of computational crystallography sessions and the 2025 Users' Meeting tutorial document shared by Sandra. Louis has been critical for the parts on geometry from BayFAI and psana related code.

As all software and the computing environments are updating all the time, please keep the timestamp of the document and the sections in mind.

This stable version is meant for viewing only. If you want to make edits/comments, or you need bleeding edge updates, you can check out the “[living](#)” version of this handbook that contains the latest fixes and updates information, but expect the content to be messy there. Content highlighted in yellow means partially completed, work in progress.

Change Log

02/03/26: Doris updated chapter “Intermediate: Prepare for real-time usage during beamtime (S3DF)” according to discussion with Fred and Pam.

01/29/26: Doris added chapters “Intermediate: Prepare for real-time usage during beamtime (S3DF)” and “Intermediate: Indexing result improvement” (mostly high-level guidance at this point) and a brief “Appendix: relevant programs and parameters”. Updated “Basic: CCTBX.XFEL Programs and Tasks” based on discussion with Pam and Vandana.

01/08/26: Doris added chapters “Intermediate: Photon energy calibration” (mostly complete, but no solutions for experiments that would throw errors), “Advanced: Event code handling”, and updated “Debugging tips”. Updated the “Within a Dataset, Defining Tasks” section to based on Selena's progress. Added chapter “Intermediate: How to make a mask”.

12/07/25: Sandra improved the chapters “Remote access to GUI” and “Initial GUI configuration for an experiment”. Doris completed the chapter “Geometry Refinement”.

11/24/25: This is the very first release as an internal test.

Helpful references

GUI source code: https://github.com/cctbx/cctbx_project/tree/master/xfel/merging

Preprint: [cctbx.xfel: a suite for processing serial crystallographic data](#)

Psana confluence page:

<https://confluence.slac.stanford.edu/spaces/LCLSIIData/pages/267391733/psana?macroName=view-file&desktop=true#psana-DetectorNames>

LUTE documentation: <https://slac-lcls.github.io/lute/dev/>

BayFAI via LUTE documentation: <https://slac-lcls.github.io/lute/dev/usage/tasks/bayfai/bayfai/>

☑ Preparation: Remote access to GUI

A note on copy paste between local computer to remote session: the short cut keys can be different so if you fail to paste something keep this in mind. For OnDemand, I couldn't copy/paste between local (mac) and the VNC session at all...

S3DF: OnDemand (least lagging, but no copy/paste)

S3DF OnDemand can be accessed through the following link:

<https://s3df.slac.stanford.edu/ondemand>

From here, navigate to 'Interactive Apps' in the top menu and select 'S3DF':

The screenshot shows the S3DF OnDemand web interface. At the top, there is a red navigation bar with 'apps' and 'My Interactive Sessions'. Below this is a breadcrumb trail: 'Home / My Interactive Sessions / S3DF'. On the left, a sidebar titled 'Interactive Apps' lists 'Desktops', 'Cryosparc', 'S3DF' (selected), 'Services', and 'Jupyter'. The main content area is titled 'S3DF' and contains the following fields:

- Description:** This app will launch an interactive graphical desktop on a compute node. This is analogous to an interactive batch job.
- Singularity Image:** A dropdown menu set to 'latest'.
- Desktop Environment:** A dropdown menu set to 'xfce'.
- Chose whether to use xfce or mate for your desktop:** A text label.
- Account:** A text input field containing 'icls:mfx100904224'.
- Slurm account to use:** A text label.
- Partition:** A text input field containing 'milano'.
- Slurm Partition to launch the Desktop on:** A text label.
- Reservation:** An empty text input field.
- Number of hours:** A spinner input field set to '2'.
- Number of hours for which the Desktop instance will run for. Note that after 30 mins of inactivity, the Desktop will automatically terminate.** A text label.
- Number of CPU cores:** A spinner input field set to '16'.
- Number of cpu cores available for Desktop instance:** A text label.
- Total Memory to allocate:** A spinner input field set to '128128'.

Once started, you might want to adjust the image quality:

S3DF (15519824)

1 node | 16 cores | Running

Host: `>_ sdfmilan104.sdf.slac.stanford.edu`

Created at: 2025-11-14 01:09:51 UTC

Time Remaining: 1 hour and 56 minutes

Session ID: 3c42dc00-46bd-4fec-b785-05490b67e5c1

Compression

Image Quality

0 (low) to 9 (high)

0 (low) to 9 (high)

Launch S3DF

View Only (Share-able Link)

Then select 'Launch S3DF'.

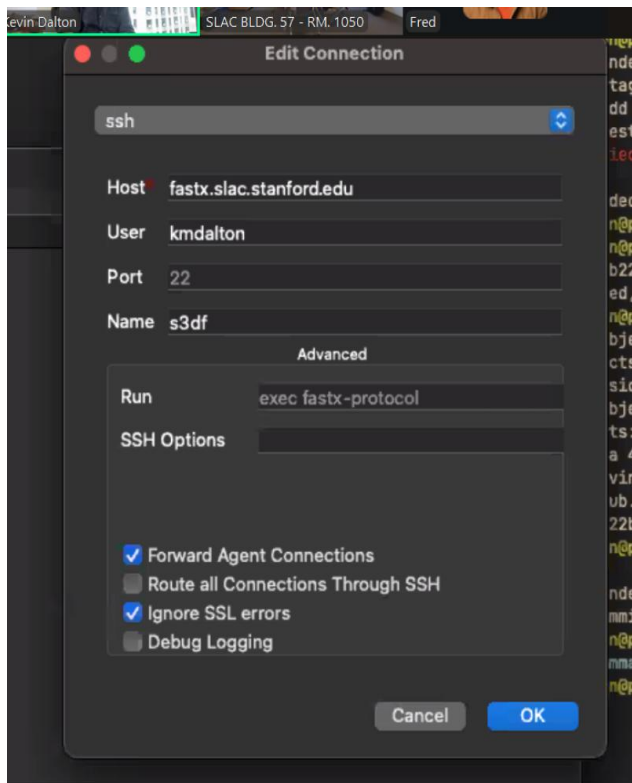
NERSC: ThinLinc

Follow the official documentation from NERSC on ThinLinc usage:

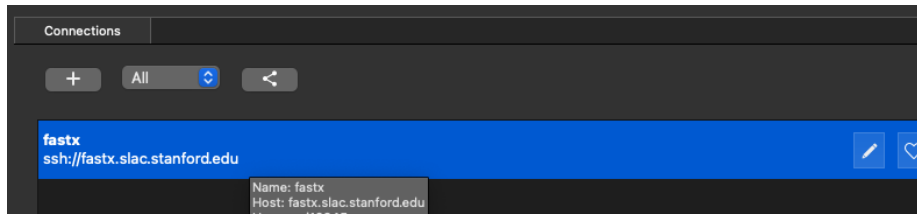
<https://docs.nersc.gov/connect/thinlinc/>

It is fairly clear and straightforward (at least on mac).

S3DF Alternative 1: FastX v4 (StarNet)



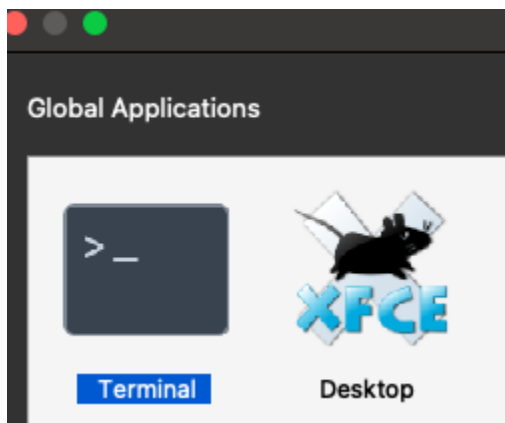
After saving the connection, double click



Put in your login credentials, you should see the usual pop up message from S3DF.

Then, click on the “+” icon on the upper left to start a new session.

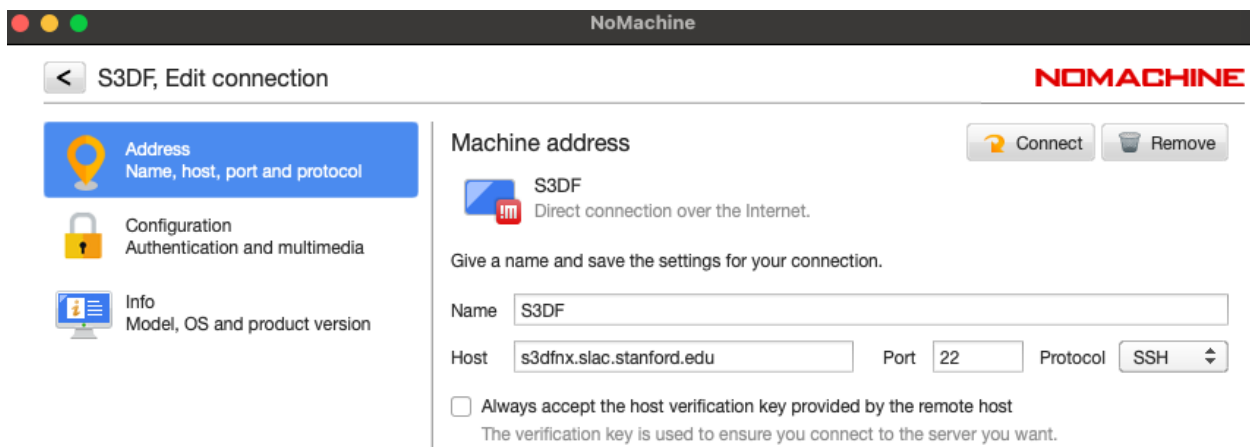
Choose Terminal rather than Desktop (might be historical reason : at some point this prevented a very ugly display issue of the GUI).



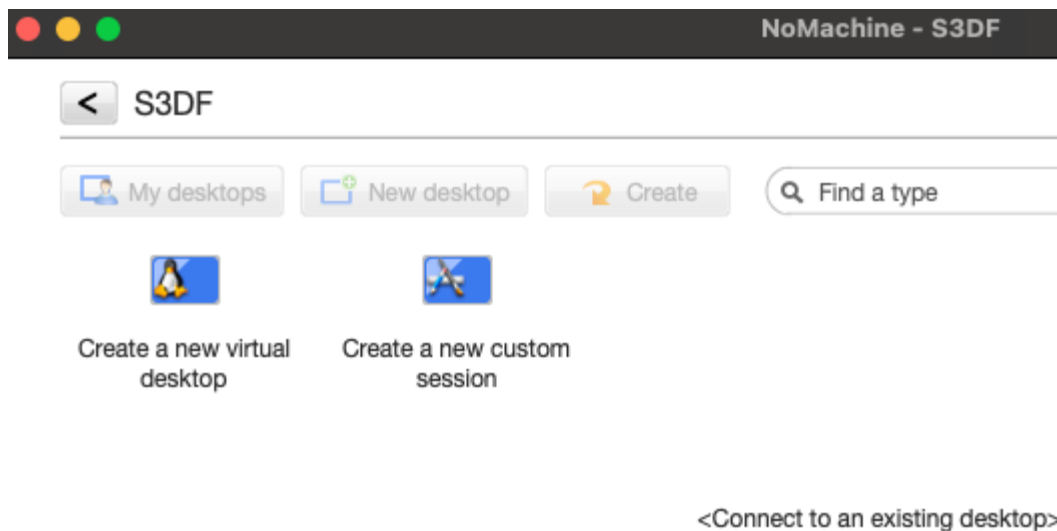
Once in the terminal, ssh into psana.

S3DF Alternative 2: NoMachine

Click “Add” to create the connection



Once saved, double click the icon for S3DF, and then double click “Create a new virtual desktop”



Then click the terminal in the desktop, once in the terminal, ssh into psana.

Other alternatives could be Xming , but might be very slow.

2. Login to the remote session via ssh

In general, to enable graphics forwarding, you want to put the -Y flag when ssh, e.g.:

```
ssh -Y <username>@s3dflogin.slac.stanford.edu
```

For S3DF, depending on your ssh config, you might need to have additional ssh into psana:

```
ssh -Y psana
```

3. Source cctbx

On S3DF

For psana2:

```
source /sdf/group/lcls/ds/tools/cctbx/psana2_setup.sh
```

For psana1:

```
source /sdf/group/lcls/ds/tools/cctbx/psana1_setup.sh
```

Tips:

- Do not recommend putting source command in `.bashrc` (too aggressive with Python).
- A good practice is to keep environment clean, not even base conda environment via `.bashrc`.

The `setup.sh` is a wrapper script of `conda_setpaths.sh` that comes with the cctbx build but also defines necessary environment variables.

⚠ The environment variables are different on S3DF vs NERSC.

If `setup.sh` is not available, make one as follows:

S3DF:

```
export SIT_DATA=/sdf/group/lcls/ds/ana/data
export SIT_PSDM=/sdf/data/lcls/ds/
export SIT_ROOT=/sdf/data/lcls/ds
export SLURM_PRIORITY_RESERVATION=lcls.onshift
umask 002
source /sdf/group/lcls/ds/tools/cctbx/build/conda_setpaths.sh
```

On NERSC

```
source /global/common/software/cctbx/alcc-recipes/cctbx/activate.sh
```

☒ Basic: Initial GUI configuration for an experiment

0. Source the right cctbx environment

See Preparation step 3 ('Source cctbx')

1. Configure experiment settings

The overall configuration of the GUI for an experiment is specified in

```
/sdf/home/<username>/.cctbx.xfel/settings.phil
```

Content in this file will pre-populate some entries in the GUI, and conversely, if you fill in or edit the corresponding textboxes in the GUI, it will edit the `settings.phil` file.

⚠ If you have never run cctbx.xfel GUI before, you won't have this folder or this file.

⚠ If you are switching between experiments, remember that GUI will overwrite `~/cctbx.xfel/settings.phil`, so it is good practice to always save a copy of the existing `settings.phil` before running.

Usually you would copy a previous `settings.phil` file as a template to start with (and make the `.cctbx.xfel` folder if it doesn't exist yet):

```
cp /sdf/home/<username>/cctbx.xfel/settings.phil <experiment_file>
```

e.g. for Summers: `/sdf/home/m/mai12345/cctbx.xfel/mfx100904224.phil`

The content of this file looks like this:

```
[mai12345@sdfiana002 ~]$ cat .cctbx.xfel/settings.phil
facility {
  name = *lcls standalone
  lcls {
    experiment = "mfxl1008021"
    web {
      location = "S3DF"
    }
  }
}
output_folder = "/sdf/data/lcls/ds/mfx/mfxl1008021/results/common/results"
mp {
  method = local lsf sge pbs *slurm shifter htcondor custom
  nnodes_index = 2
  nnodes_tder = 1
  nnodes_scale = 1
  nnodes_merge = 1
  queue = "milano"
  extra_options = "--account=lcls:mfxl1008021"
  env_script = "/sdf/data/lcls/ds/prj/prjlumine22/results/cctbx/setup.sh"
  phenix_script = "/sdf/group/lcls/ds/tools/conda_envs/dorismai/rs_setup.sh"
}
experiment_tag = "common"
db {
  host = "172.24.5.182"
  name = "mfxl1008021"
  user = "mfxl1008021"
  password = "lcls"
}
```

It is ok to start the GUI from scratch or from a partially filled `settings.phil` file. But if you did put down the “output_folder”,

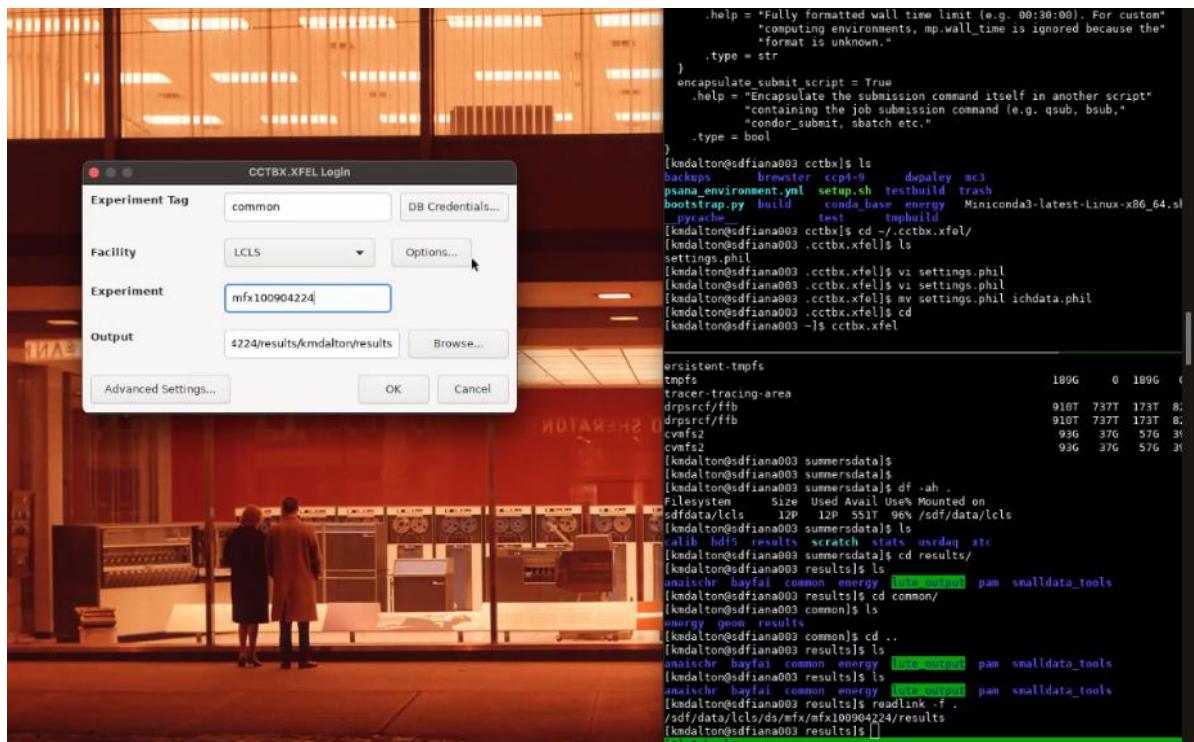
⚠ Make sure paths already exist, as the GUI will not create new folders and instead just crashes or freezes with terminal output errors.

2. Start GUI

Run: `cctbx.xfel` in terminal. Launching `cctbx` can take a minute.

When you launch the GUI using this PHIL `phil`, you will see the following window and you can spot some of the corresponding entries such as “Experiment Tag” (`experiment_tag`), “Output” (`output_folder`), “Facility”(facility.name), and “Experiment” (`facility.lcls.experiment`) from the setting.phil file.

Fill fields if not already populated by the experiment settings.phil file:



The output field in the text box above is:

`/sdf/data/lcls/ds/mfx/mfx100904224/results/kmdalton/results`

Experiment Tag and Output Folder

Conventionally, the experiment tag and your process folder (relative path to the experiment folder) should match, but it looks like this is not strictly enforced. In this example, we have an experiment tag as “common”, but the processing folder (relative path) can be “pam” (used in the examples below) or “kmdalton” (in the screenshot above).

Unless you are the official processing person (in which case the output folder should be “common” (relative path)), you are recommended to make a folder for yourself (e.g. named “pam”).

The overall experiment folder is:

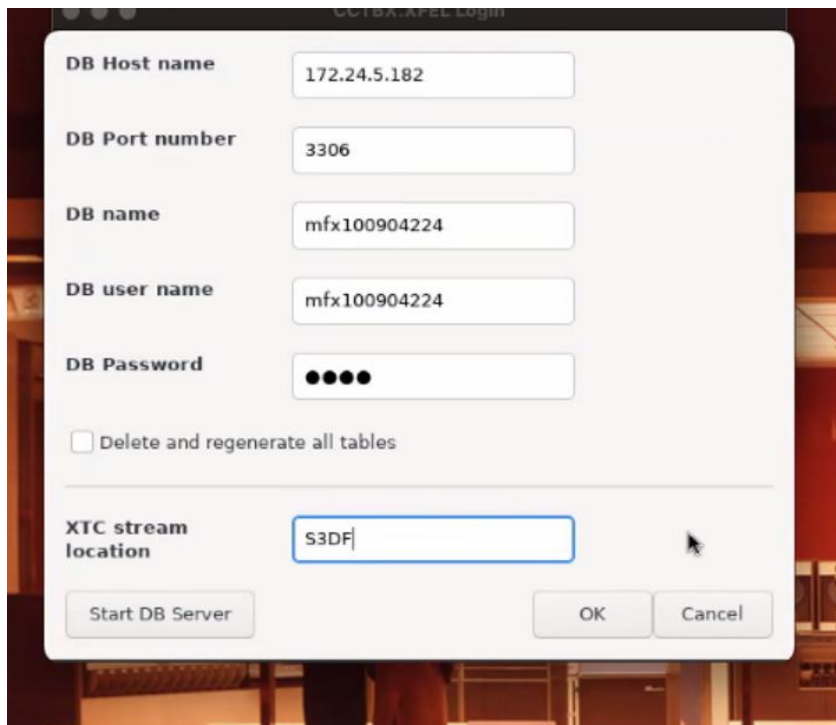
```
[pam@sdfiana023 results]$ pwd
/sdf/data/lcls/ds/mfx/mfx100904224/results
```

And the GUI expects your processing folder (absolute path) would be the following:

```
output_folder = "/sdf/data/lcls/ds/mfx/mfx100904224/results/pam"
```

DB Credentials

Modify DB credentials as needed, this corresponds to the “db” section in the settings.phil file. Name and user are usually the experiment.



Password: lcls

⚠ The buttons at the bottom of the DB window sometimes disappear (for example, when running cctbx.xfel via OnDemand). You can exit out of the window by using the return or escape key on your keyboard.

Options:

Leave them unselected.

Advanced Settings

Modify advanced settings as needed -- this corresponds to the “mp” section of the settings.phil file.

⚠ Make sure that the “Environment setup script” matches with the script you used in step 0 to source cctbx.

Most of the other entries are related to slurm job configuration, including the “Extra submission arguments” (which corresponds to “extra_options” under “mp” as a string). Milano has 128 cores and WEKA takes 8 cores, use no more than 120 here as number of processors per node.

The slurm account in Extra submission arguments” should match the experiment you are processing.

⚠ If real-time processing during beamtime, you can add “--reservation=lcls:onshift”, but use it wisely: the job priority is higher but you are restricted to 10 nodes.

The screenshot displays the CCTBX XFEL GUI with two main sections: Multiprocessing Options and Data Analysis Options. The Multiprocessing Options section includes a dropdown for Multiprocessing (set to 'slurm'), a text field for Queue (set to 'milano'), a checkbox for Number of processors per node (set to 'Auto' with a value of 120), a text field for MPI command (set to 'mpirun'), a text field for Environment setup script (set to '/sdf/group/lcls/ids/tools/cctbx/build/conda_setpaths.sh'), and an empty text field for Phenix setup script. Below these are controls for Nodes per job: Indexing (1), TDER (1), Scaling (1), and Merging (1), each with minus and plus buttons. An Extra submission arguments field contains '--account=lcls:mfx100904224'. The Data Analysis Options section has a dropdown for Processing back end (set to 'cctbx.xfel') and a description: 'Process the data according to Brewster 2018, using DIALS for indexing, refinement and integration, with stills-specific defaults. Reads image files directly.' At the bottom right are 'Cancel' and 'OK' buttons. To the right of the GUI, a terminal window shows a series of commands and outputs, including 'cctbx.xfel ls', 'cctbx.xfel cd -/...', and 'cctbx.xfel ls'.

Multiprocessing Options

Multiprocessing: slurm

Queue: milano

Number of processors per node: ☐ Auto 120 - +

MPI command: mpirun

Environment setup script: /sdf/group/lcls/ids/tools/cctbx/build/conda_setpaths.sh

Phenix setup script:

Nodes per job

Indexing: 1 - + TDER: 1 - + Scaling: 1 - + Merging: 1 - +

Extra submission arguments --account=lcls:mfx100904224

Data Analysis Options

Processing back end: cctbx.xfel

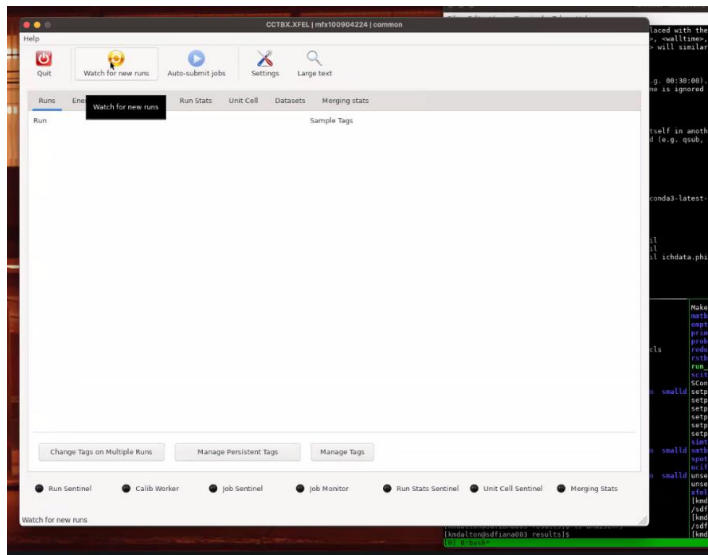
Process the data according to Brewster 2018, using DIALS for indexing, refinement and integration, with stills-specific defaults. Reads image files directly.

Cancel OK

3. Populating runs

In GUI, click **Watch for new runs** (Hold until button turns yellow.) The buttons can be a bit laggy.

⚠ At most one person (instance of GUI) can watch at any time, so it is good practice to turn the watch off when not needed. For reprocessing, you only need to turn it on for a few seconds to load all the run information once.



You should see runs populated under the “Run” page.

4. Tagging runs

To add a tag, click “Manage Tags” at the bottom.

To associate an added tag for multiple runs, click “Change Tags on Multiple Runs” at the bottom.

For real-time processing, it is helpful to automatically tag an incoming run if you know the sample condition won’t change for a while. In this case, click “Manage Persistent Tags” at the bottom.

⚠ if GUI crashes during real-time processing, when you relaunch the GUI, you need to configure this persistent tagging again. Otherwise before you know there will be many untagged runs coming in.

Basic: CCTBX.XFEL Programs and Tasks

1. Defining Trial (i.e. Indexing parameters)

Go to the “Trials” page of the GUI.

This is a somewhat peculiar part of how the GUI is organized. When creating a trial, what you put in the big textbox is actually parameters for the Indexing task. However, all the other tasks are defined under Datasets (see step 2).

The format/grammar of the content is the same as the phil file, which stands for Python Hierarchical Input Language (a kind of format used for cctbx and phenix programs):

https://cci.lbl.gov/docs/cctbx/doc_low_phil/

Under the hood, the Indexing task is performed by calling the program `dials.stills_process`. If you don't know what parameters are accepted by this program to put in the textbox, you can use `dials.stills_process -ca 2 -e 10`

Similar command works for other cctbx/dials programs if you want to learn more about the parameters, and the flags specify the expert (verbose) level of the help messages.

An example input looks like this:

```
dispatch {
  hit_finder {
    minimum_number_of_reflections = 16
  }
}
spotfinder {
  filter {
    min_spot_size = 2
    max_spot_size = 100
  }
  threshold {
    algorithm=dispersion
    dispersion {
      gain = 1
      kernel_size = 1 1
      global_threshold = 10
    }
  }
}
indexing {
  stills.reflection_subsampling.enable=True
```

```

known_symmetry {
  space_group = P 1 21 1
  unit_cell = 87.970 52.420 95.650 90 95.609 90
}
refinement_protocol {
  d_min_start = 2
}
multiple_lattice_search {
  max_lattices = 3
}
}

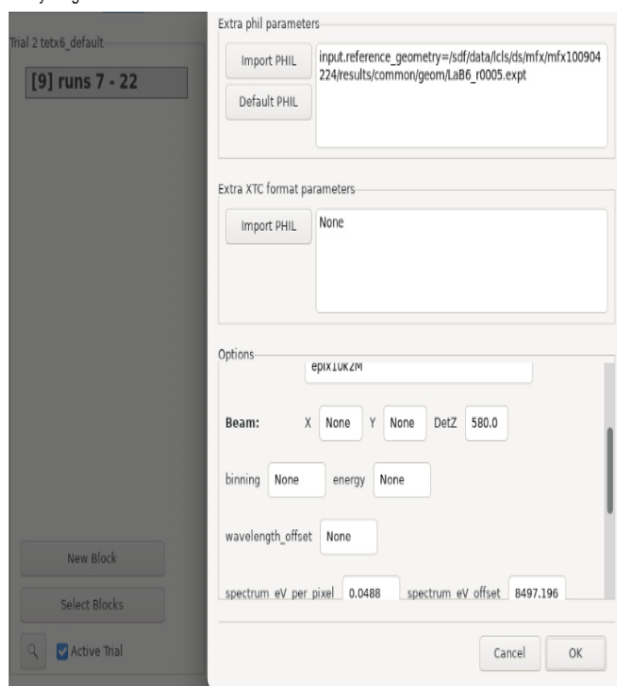
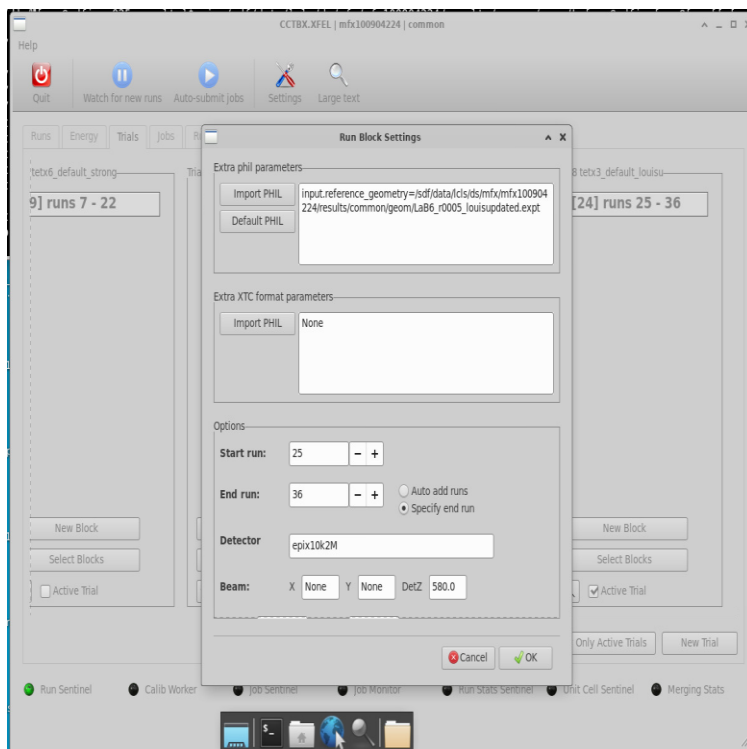
```

Remember to check space group and unit cell information here (and make new trials if necessary) if you have samples with distinct values.

⚠ During real-time processing, it could be helpful to add the line `integrate = False` within the `dispatch` block (above `hit_finder`) to speed up the indexing results. However, remember to either re-run (by making another trial) without this flag or perform ensemble refinement to make sure you can eventually do scaling and merging (See Section 3. Within a Dataset, Defining Task)

Within a trial, define Run Group

- You need to specify the start and end (inclusive) runs. For real-time processing, “Auto add run” can be helpful, but you need to remember terminate it by specifying the end run one sample is changed.
- A path to .expt file that specifies the geometry (see step a below) at the top textbox.
- Under the second large text box “Extra XTC format parameters”, for psana2, you need to put the line “mode=psana2_idx”.
- Specify the detector name. If you don’t know exactly the format, the `detnames` program from psana is helpful (see the Debug chapter).
- Specify binning, if any
- Put in the `spectrum_eV_per_pixel` and `spectrum_ev_offset` values if you have that (see the Photon energy calibration chapter). You can either manually input them, or if you have saved the .phil from photon energy calibration, or use the “Import PHIL” button to populate the values from the file for you.
- Load the pixel mask (see step b). If you don’t know how to make mask, see the corresponding chapter.



a. For each run group specify the geometry file

For the example experiment we look at, the detector calibration was performed using LaB6. The experiment file can be found [here](#):

/sdf/data/lcls/ds/mfx/mfx100904224/results/common/geom/LaB6_r0005_louisupdate
d.expt

This will be input.reference_geometry in the run block settings

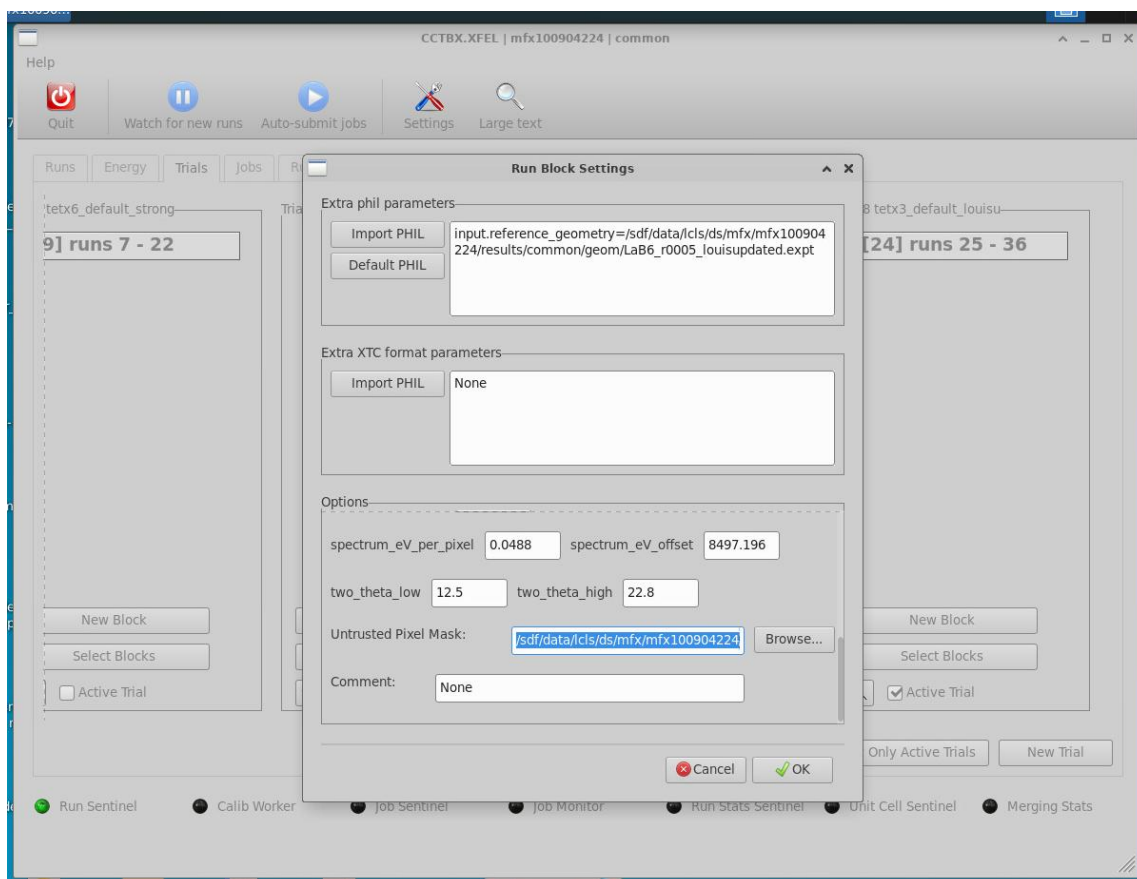
The .expt file will overwrite the values in the text boxes below for “Beam: X Y DetZ”

b. For each run group load pixel mask

For the summers experiment a mask can be found here:

/sdf/data/lcls/ds/mfx/mfx100904224/results/common/pixels.mask

This can be used in the following



c. Extra XTC format parameters

Other than specifying mode for psana2, if there are any issues of the XTC files, this textbox is likely where you need to modify, such as specifying spectrum address, putting a wavelength fallback value when eBeam is missing etc. Content in this textbox will be part of the data.loc file that enables retrieving and viewing the images.

d. Check this trial as the active trial, and uncheck other if you previously have other processing.

2. Defining Datasets

Go to the “Datasets” page of the GUI.

Refer to “Tagging runs” in the previous chapter step 3 “Populating runs” if you have no clue about tags.

Depending on how you tag the runs, select if you want the intersection or the union of the tags to process a dataset. Intersection makes sense if say you have one tag of mixing time and another tag for temperature. Union makes sense if you want to combine runs that are similar (e.g. tagged by different delivery speed but otherwise the same sample).

Not all runs defined by the tags you choose will be processed – only runs within the run groups of the trial will be used to look at tags. For example, run 1-40 are all tagged as apo, but your run group under the trial is defined as run 20-40, then the apo dataset will only process run 20-40. See below on task definition to see where trial comes in.

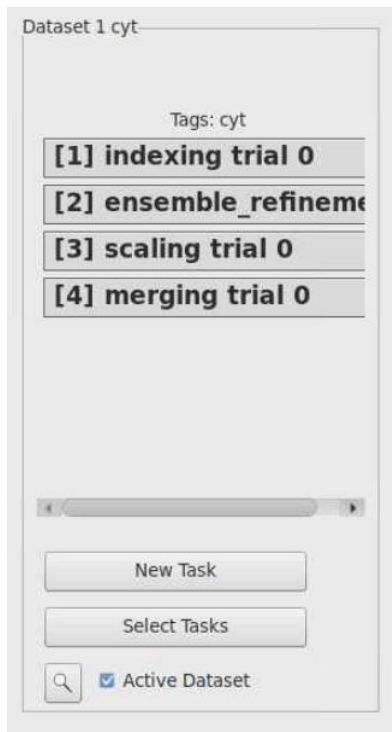
The name of the dataset will become the name of the folder for the Merging task.

 **Do NOT contain space or weird characters in the dataset name, as merging can fail. Use a single word (linked by underscore is fine).**

Check this dataset as the active dataset, and uncheck other if you previously have other processing.

3. Within a Dataset, Defining Tasks

Each task has to be associated with a specific trial. Under a dataset, click “New Task” to add a task and specify the trial and the parameters (PHIL format) for that task. Typically, you would have the following 4 tasks to run for a dataset:



The tasks you define are chained in the listed order, such that for a particular run, a local task (e.g. scaling, see explanation below) won't run until the previous local task finishes. Ensemble refinement is an optional task, so you could just chain a scaling task right after the indexing task.

⚠ You might notice that in practice, after you define the trial and run groups, with the "Auto submit jobs" button on, indexing jobs can be automatically submitted for those runs, even if you haven't defined the Indexing task.

⚠ That said, you must define the Indexing task explicitly in the Dataset and chain the other tasks (such as shown in the screenshot above) in order to run the other tasks. For example, you cannot leave only Scaling and Merging tasks there, even if you have already got Indexing and Ensemble refinement results from previous processing.

⚠ Technically you can chain Indexing task directly to Scaling task. However, make sure you did NOT set `integrate=False` for Indexing (which, remember, is set when you define the trial and cannot be edited).

Task Output Folder Structures

Local task

Indexing, ensemble refinement, and scaling are all considered local task, meaning the task is performed at the level of each run, and the outputs are organized under each run. A typical run folder structure (relative to experiment folder path) is this:

r0023/002_rg015

Which means the output is for **run 23** processed from indexing parameters in **trial 2** and other information (e.g. geometry and pixel mask and energy calibration) associated with **run group 15**.

The non-indexing local task has its own output folder named by the task id its assigned by the GUI, e.g. r0023/002_rg015/task009

Global task

Merging, however, is a global task as it is performed across runs. The output folder is named by the dataset name, and lives in the experiment folder where each run folder lives. This task will be run for many versions, as the previous local task (scaling) finishes for more and more runs. See the “Outputs” subsection under the “Merging” section below for more details.

Indexing

See step 1 for trial definition. You don’t need to put any parameters here.

Outputs:

Within the run folder, you would have a data.loc file, out folder, stdout folder, a params_1.phil file, and slurm job submission script submit.sh and its wrapper submit_submit.sh.

The out folder contains the .expt and .refl outputs. One idx-{number}_refined.expt file does NOT necessarily mean 1 image – it could contain multiple images:


```

"imageset": [
  {
    "__id__": "ImageSet",
    "images": [
      "/sdf/data/lcls/ds/mfx/mfx100904224/results/pam/r0018/009_rg032/data.loc"
    ],
    "single_file_indices": [
      1628
    ],
    "mask": null,
    "gain": null,
    "pedestal": null,
    "dx": null,
    "dy": null,
    "params": {}
  },
  {
    "__id__": "ImageSet",
    "images": [
      "/sdf/data/lcls/ds/mfx/mfx100904224/results/pam/r0018/009_rg032/data.loc"
    ],
    "single_file_indices": [
      52348
    ],
    "mask": null,
    "gain": null,
    "pedestal": null,
    "dx": null,
    "dy": null,
    "params": {}
  }
],

```

The “single_file_indices” tells you the actual index of the image. In other words, if you are viewing all images (including non-hit) of the entire run using `dials.image_viewer data.loc load_models=False`, you can jump to view the same image using this index (+1, apparently 0-indexed).

To just view some example of indexed images, you could do `dials.image_viewer idx-{whatever number}_*. {expt, refl} load_models=False`

Errors of the job would end up in the `stdout` folder although it is very hard to debug (especially mpi related) errors based on messages there. I usually only look at the `err.out` file there.

The parameters you put under trial definition would end up in `params_1.phil`.

“Extra XTC format parameters” would end up in `data.loc`, which used by the GUI to talk to psana and get images data.

See the chapter “Intermediate: Indexing result improvement” for more details on checking the indexing statistics and strategies for improvement.

Ensemble refinement

Refines the geometry, unit cell, etc. and performs integration.

Example inputs

```
combine_experiments.clustering.use=False
reintegration.integration.debug.output = True
reintegration.integration.debug.separate_files = False
reintegration.integration.mp.nproc = 32
reintegration.integration.summation.detector_gain = 0.46
```

You may want to change the `detector_gain` in the last line depending on the type of detector (e.g. 1 used for Epix10ka2M).

Outputs:

Within the task output folder (e.g. `r0023/002_rg015/task009`), it has another folder like `combine_experiments_t002`, where `t002` means **trial 2**. Within this folder, you two folders `final_extracted` and `intermediates`. The integration results (`.expt` and `.refl` files) as well as log files are in the `intermediates`.

Scaling

Scaling and Merging both use `cctbx.xfel.merge` program, and it's the dispatcher list that decides what processing is actually performed. An example phil parameters for scaling is attached below. As you can see, "scaling" is there, but it really is just one of many steps needed to perform for the Scaling task.

Example inputs

```
dispatch.step_list=input balance model_scaling modify filter
errors_premerge scale postrefine statistics_unitcell statistics_beam
model_statistics statistics_resolution
input.parallel_file_load.method=uniform
filter.outlier.min_corr=-1
filter.algorithm=unit_cell
filter.unit_cell.value.relative_length_tolerance=0.01
select.algorithm=significance_filter
select.significance_filter.sigma=0.1
#select.significance_filter.min_ct=200
#select.significance_filter.max_ct=300
scaling.model=/sdf/data/lcls/ds/mfx/mfxl1008021/results/common/models/
L10080_no_mix_rsich_refine_6.pdb
scaling.resolution_scalar=0.96
merging.d_min=1.2
```

```
merging.merge_anomalous=True
postrefinement.enable=True
statistics.n_bins=20
output.save_experiments_and_reflections=True
```

Outputs:

The task output folder is structured very similar to the indexing output folder (one level up of this task output folder), except that there is not data.loc file here.

Merging

Example inputs

```
dispatch.step_list=input model_scaling statistics_unitcell
statistics_beam model_statistics statistics_resolution group
errors_merge statistics_intensity merge statistics_intensity_cxi
publish
input.parallel_file_load.method=uniform
scaling.model=/sdf/data/lcls/ds/mfx/mfxl1008021/results/common/models/
rsich_high_dms0_150ms_mixing.pdb
scaling.resolution_scalar=0.96
statistics.n_bins=20
merging.d_min=1.28
merging.merge_anomalous=True
merging.error.model=mm24
```

To upload merged .mtz files into google drive, refer to this instruction to set up (https://github.com/cctbx/cctbx_project/tree/master/xfel/merging/application/publish), and add two more lines in the inputs:

```
publish.drive.credential_file=
publish.drive.shared_folder_id=
```

Additional notes

⚠ If you don't specify the google drive related parameters to publish mtz files but you have `publish` in your `dispatch.step_list`, the merging job will finish with a status "EXIT" which normally indicates something went wrong with the job, but here it could be fine. You can check the `err.log` the output folder to confirm it is this type of error that led to EXIT, and you should still have all the other proper output files.

⚠ If you are looking at some old processing data, you might see phil parameters like `merging.error.model=ev11` `merging.error.ev11.algorithm=`. This was prototype (deprecated) code and is transitioned to mm24 now.

Outputs:

Within the output folder (named after the dataset, lives under the experimental folder), there are many versions as more runs coming in. For final result, you want to look at the folder with the largest version number. Within this folder, the key files are:

- {dataset_name}_v{version_number}_main.log: contains merging stats
- {dataset_name}_v{version_number}_all.mtz: overall merged output
- {dataset_name}_merging{task_id}_v{version_number}_params.phil: your input parameters
- {dataset_name}_merging{task_id}_v{version_number}_submit.sh

There are also a lot of other logging files, including log.out and err.out, and wrapper submit script for slurm.

For quick triage I would mainly focus on the tables shown below in the ...main.log file. The tables for all (rather than for odd/even) are closer to the end of the file. I would look at $\text{obs_multi} > 10$, $\text{CC1/2} > 0.3$, and other metrics to decide resolution and if any pathology.

Intensity Statistics (all accepted experiments)												
Total reflections (I == 0.0):			34									
Total reflections (I > 0.0):			7517841									
Total reflections (I < 0.0):			1751847									
Resolution-limited reflections (I == 0.0):			32									
Resolution-limited reflections (I > 0.0):			7455784									
Resolution-limited reflections (I < 0.0):			1706068									
Bin	Resolution Range	Completeness	%	<asu multi>	<obs multi>	n_meas	asu_n_meas	Merged <I>	Merged <I/sig(I)>	Unmerged mean(I/sig(I))	Unmerged stddev(I/sig(I))	Unmerged skew(I/sig(I))
1	-1.0000 - 3.5287	[5458/5473]	99.73	293.86	294.67	1608298	5458	248032	26.442	41.850	71.354	2.822
2	3.5287 - 2.8008	[5243/5243]	100.00	192.61	192.61	1009869	5243	107071	18.601	24.233	39.499	3.536
3	2.8008 - 2.4467	[5220/5220]	100.00	162.16	162.16	846470	5220	49774	15.187	13.603	21.679	3.711
4	2.4467 - 2.2230	[5177/5177]	100.00	144.20	144.20	746499	5177	33207	13.237	9.867	15.504	3.774
5	2.2230 - 2.0636	[5139/5139]	100.00	131.78	131.78	677218	5139	22097	11.462	6.984	11.052	3.843
6	2.0636 - 1.9419	[5149/5149]	100.00	121.00	121.00	623040	5149	13654	9.730	4.690	7.700	3.921
7	1.9419 - 1.8447	[5127/5127]	100.00	110.40	110.40	566007	5127	8321	8.040	3.165	5.415	3.808
8	1.8447 - 1.7644	[5138/5138]	100.00	101.79	101.79	523021	5138	5124	6.404	2.140	3.875	3.542
9	1.7644 - 1.6964	[5117/5117]	100.00	94.64	94.64	484293	5117	3570	5.314	1.568	3.003	3.037
10	1.6964 - 1.6379	[5095/5095]	100.00	83.48	83.48	425309	5095	2639	4.410	1.247	2.470	2.099
11	1.6379 - 1.5867	[5098/5098]	100.00	64.34	64.34	328013	5098	2122	3.457	1.032	2.168	1.563
12	1.5867 - 1.5413	[5096/5096]	100.00	54.46	54.46	277541	5096	1707	2.825	0.856	1.978	1.050
13	1.5413 - 1.5007	[5089/5089]	100.00	45.67	45.67	232406	5089	1346	2.226	0.695	1.832	0.690
14	1.5007 - 1.4641	[5103/5103]	100.00	35.72	35.72	182268	5103	1208	1.865	0.637	1.769	0.540
15	1.4641 - 1.4308	[5107/5107]	100.00	31.06	31.06	158606	5107	963	1.506	0.517	1.670	0.219
16	1.4308 - 1.4004	[5060/5060]	100.00	27.09	27.09	137095	5060	818	1.247	0.453	1.639	0.127
17	1.4004 - 1.3724	[5078/5079]	99.98	20.70	20.70	105124	5078	765	1.020	0.421	1.629	0.090
18	1.3724 - 1.3465	[5088/5088]	100.00	17.69	17.69	90022	5088	697	0.879	0.391	1.608	0.071
19	1.3465 - 1.3224	[5069/5069]	100.00	15.77	15.77	79959	5069	638	0.757	0.354	1.610	0.075
20	1.3224 - 1.3000	[5014/5018]	99.92	12.12	12.13	60821	5014	722	0.704	0.395	1.657	0.099
All		[102665/102685]	99.98	89.22	89.24	9161879	102665	26175	6.880	0.000	0.000	0.000

CC 1/2

CC 1/2	
5458/5473	0.992593
5243/5243	0.990634
5220/5220	0.985961
5177/5177	0.982737
5139/5139	0.979833
5149/5149	0.978051
5127/5127	0.972897
5138/5138	0.965396
5117/5117	0.955690
5095/5095	0.933101
5098/5098	0.890780
5096/5096	0.841486
5089/5089	0.760742
5102/5103	0.688548
5106/5107	0.502063
5053/5060	0.398154
5048/5079	0.263410
5054/5088	0.231096
4999/5069	0.161677
4739/5018	0.149397
102247/102685	0.995119

Quick Workflow Checklist

1. Connect via **FastX**
2. `ssh -Y` into S3DF
3. Source env:


```
source /sdf/group/lcls/ds/tools/cctbx/setup.sh
```
4. Copy `.phil` settings
5. Start GUI → enter experiment info
6. Configure DB credentials
7. Set options + advanced settings
8. Start processing (watch new runs or manual selection)
9. Geometry calibration and energy calibration, in either order

10. Make/load Pixel Mask
11. Define trial, run groups if not already
12. Define dataset
13. Start an indexing task (or more)

Intermediate: Prepare for real-time usage during beamtime (S3DF)

Before the beamtime

General

- Check that the experiment folder has been set up on S3DF and visible on eLog
- Collect Dark runs (test takepedcs/makepedcs)
- Collect calibrant data and run BayFAI (during commissioning)

Note on BayFAI / geometry – 2026/02

See LUTE documentation on how to set up and run BayFAI to estimate geometry:

<https://slac-lcls.github.io/lute/dev/usage/tasks/bayfai/bayfai/>

As of 2026/02, this involves following the instructions linked above in the experiment folder. After the set up, you should see the BayFAI workflow auto-populated on elog --> Workflow --> Definitions, and you can adjust it appropriately. For example, you want to change the Trigger to MANUAL rather than the end of each run, and maybe adjust slurm arguments like --ntasks as needed.

Database

Make sure the users have requested to set up MariaDB on their form to use cctbx.xfel GUI.
If this step is missing, you need to email pcds-datamgt-l to set it up.

If you try to launch the GUI and see the following error, then DB is not set up properly yet.

```
(base) [fpoitevi@sdfiana024 .cctbx.xfel]$ cctbx.xfel
Traceback (most recent call last):
  File "/sdf/group/lcls/ds/tools/cctbx/psana2/build/../modules/cctbx_project/xfel/ui/command_line/xfel_gui_launch.py", line 42
    , in OnInit
        if self.frame.connect_to_db(drop_tables=self.login.drop_tables):
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/sdf/group/lcls/ds/tools/cctbx/psana2/modules/cctbx_project/xfel/ui/components/xfel_gui_init.py", line 1373, in connect
to_db
    self.db = xfel_db_application(self.params, drop_tables = drop_tables, verify_tables = True)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/sdf/group/lcls/ds/tools/cctbx/psana2/modules/cctbx_project/xfel/ui/db/xfel_db.py", line 406, in __init__
    dbobj = get_db_connection(params)
             ^^^^^^^^^^^^^^^^^^^^^^
File "/sdf/group/lcls/ds/tools/cctbx/psana2/modules/cctbx_project/xfel/ui/db/__init__.py", line 91, in get_db_connection
    raise e
File "/sdf/group/lcls/ds/tools/cctbx/psana2/modules/cctbx_project/xfel/ui/db/__init__.py", line 72, in get_db_connection
    dbobj=MySQLdb.connect(
           ^^^^^^^^^^^^^^^
File "/sdf/group/lcls/ds/tools/cctbx/psana2/lib/python3.12/site-packages/MySQLdb/__init__.py", line 121, i
n Connect
    return Connection(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/sdf/group/lcls/ds/tools/cctbx/psana2/lib/python3.12/site-packages/MySQLdb/connections.py", line 195
, in __init__
    super().__init__(*args, **kwargs)
MySQLdb.OperationalError: (1045, "Access denied for user 'mfxf101251025'@'192.168.101.2' (using password: YES)")
OnInit returned false, exiting...
(base) [fpoitevi@sdfiana024 .cctbx.xfel]$ history | grep mysql
```

You can also verify this with mysql command:

```
mysql -h 172.24.5.182 -u <experiment> <experiment> -p
```

See the Debugging chapter regarding Database for more mysql commands and info.

GUI readiness

Some test runs (energy calibration) should be done before the beamtime to test GUI basic functionality.

- Can you successfully launch the GUI? Check that remote access works, and sourcing the cctbx environment works.
- Can the GUI access MariaDB and you see the runs when clicking “Watch for new runs”?
- Can you do some fake jobs just to see if they can even be submitted/run? Like averaging a run and visualize the averaged image?

Analysis info

- Unit cell and space group, and a reference PDB file from users
- A template of phil parameters (at least for the Indexing task/trial definition), might be from users or from a recent beamtime
- Optionally a reference geometry .expt file, potentially from a recent beamtime, if not using BayFAI
- Optionally a reference mask file, potentially from a recent beamtime

During the beamtime

The main question to answer during beamtime is have enough data been collected to move on. Ideally, you want to proceed all the way to merged results and look at the stats there. In practice, many difficulties can come up, and a quick ballpark can come from the number of indexed images.

Make initial files for indexing

If you don't have a geometry file yet, start with averaging a run and create one.

If you don't have a mask yet, start with an averaged run and make one.

It is unlikely that your initial files and phil parameters are good enough. Typically then you need to perform energy calibration, refine the geometry, update the mask, and maybe even explore the phil parameters for spot finding etc. This means that indexing (different trials) and ensemble refinement tasks could be processed many times in slight variations, just like how you would have run the GUI for offline reprocessing.

Hit/indexing progress

You want to monitor “Run Stats” and “Unit Cell” tabs to look the rate and the number of successfully indexed images. Some users might have a rough estimate of how many indexed crystals they need.

You might find the following tricks helpful to speed up the processing:

- Set up persistent tag to automatically tag incoming runs (but remember to change it when sample changes).
- Set integrate=False when defining the Trial to perform just indexing without integration. Remember to get rid of this or perform ensemble refinement to eventually do scaling/merging.

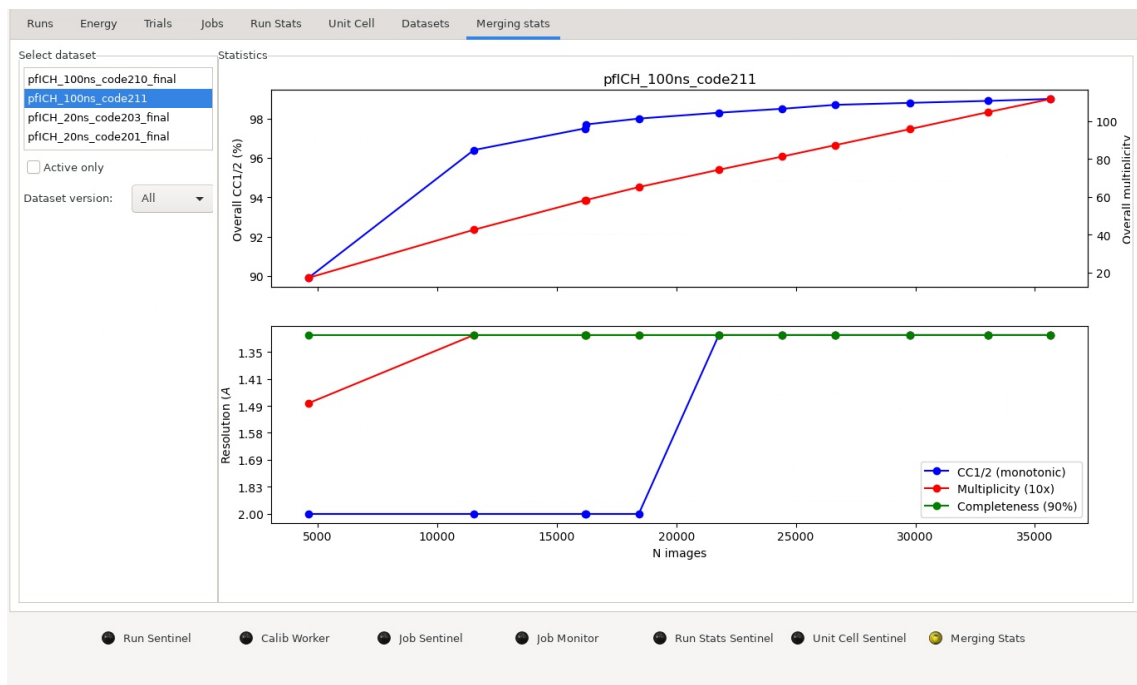
Merging progress

If things are going well and we are beyond indexing and integration, next we can look at scaling and merging. You want to look at the “Merging stats” tab:

The “Active only” refers to the dataset, and you can untick it or not. Leave the Dataset version as “All” and you should see how the statistics evolve over the versions (the version number increases as more images come in).

 **Do NOT select a particular version of the dataset. The plotting will fail (the sentinel “Merging Stats” on the bottom right turns red) and you might need to switch to other tabs and then come back to refresh the GUI out of a failed status, or relaunch the GUI.**

 Note that it can take a while to plot the image, and the main way to tell if the plotting is hanging/stuck or not is to look at the sentinel. If after a few seconds of no update and the sentinel hasn’t changed to yellow (working) (or it has changed to red), it might be stuck.



You want to see the metrics increase monotonically and plateau as you collect sufficient data. Make sure to not lump together different samples under a dataset as that could mess up the stats trajectories.

The purpose of this tab is to gauge if more data need to be collected. To see a more comprehensive and details set of merging statistics, you should go to the ...main.log file in the merging output folder.

☑ Intermediate: Geometry Refinement

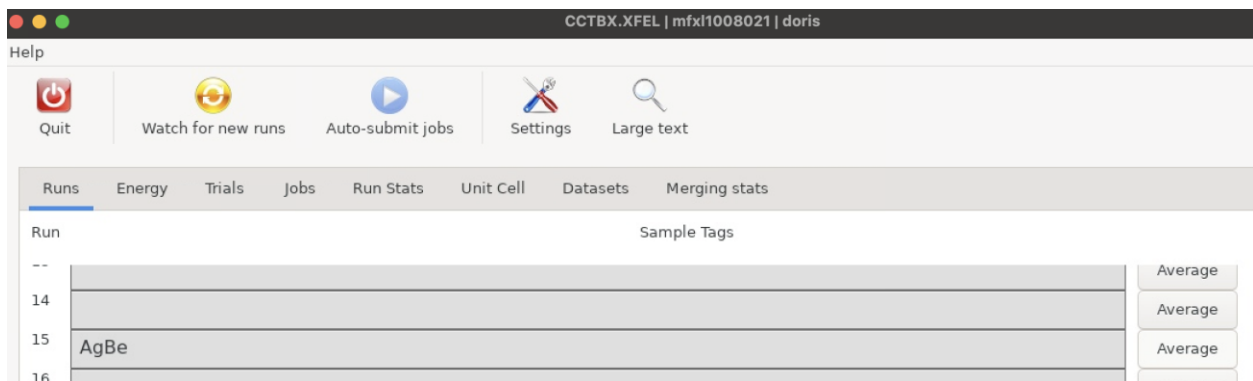
cctbx/dials understands the geometry through psana. If BayFAI was used to estimate detector distance, it will push the result to a database such that psana2 Datasource can retrieve information from it, and this is how cctbx will understand the initial geometry. For psana1, this depends on the file written by BayFAI to the calib folder of the experiment: `/sdf/data/lcls/ds/mfx/{experimentname}/calib`. This file is still written for psana2 BayFAI but is just for logging purpose as the database is the way to communicate with cctbx now.

Without BayFAI, psana use the default metrology data.

Obtaining an initial geometry .expt file

Average a run

If cctbx can retrieve images and geometry from psana properly, you would first click “Average” for the calibrant run to make powder:



Under the hood, averaging a run uses geometry from psana, which could come from BayFAI or default metrology.

The status of the average job is not checked in GUI under jobs, but actually in the terminal like your regular slurm job submission.

Once averaging is done, go to the output folder, should be output path like:

`/sdf/data/lcls/ds/mfx/{experiment_name}/results/{experiment_tag}/averages/000`

If you see more folders like 000, 001, ... these are different versions of averaging done for this run. Within a version folder, you should have `data.loc`, `submit.sh`, `submit_submit.sh` (wrapper submit script), and the `out` and `stdout` folders. In the `out` folder, you should have `max.cbf`, `std.cbf`, and `avg.cbf`. I find it easier to work with `max.cbf` for geometry stuff, and `std.cbf` for masking.

Generate an initial .expt file

In the terminal at the out folder:

```
dials.import max.cbf wavelength=1.4443
```

By default it generates a file named imported.expt.

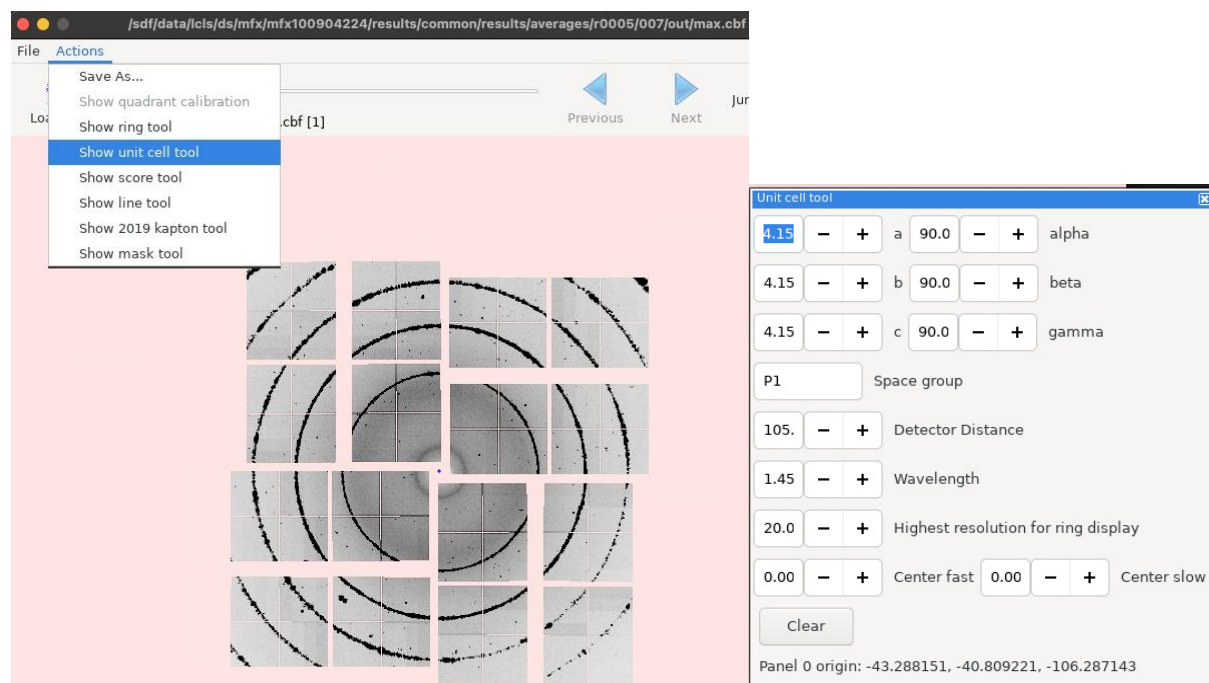
Visualize in dials.image_viewer the geometry fit

Launch image: `dials.image_viewer imported.expt`

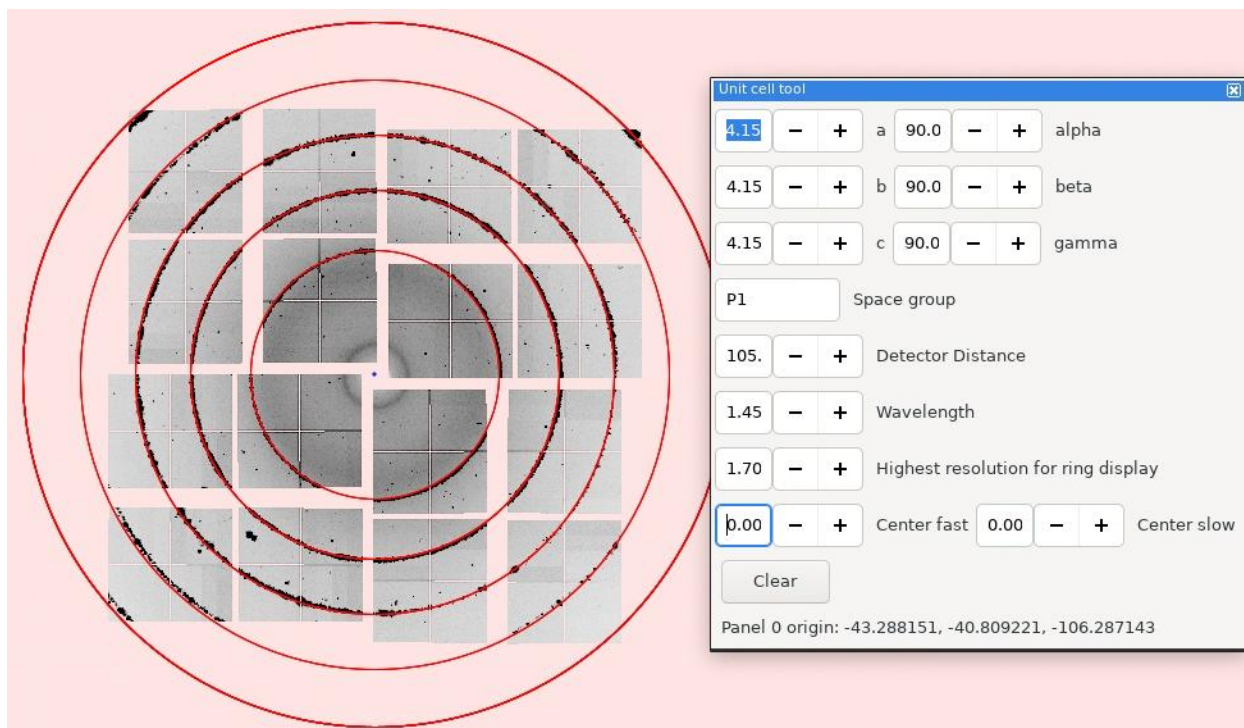
If the calibrant is LaB6 rather than Agbe, add the following to the command above:

```
unit_cell=4.15,4.15,4.15,90,90,90
```

After you have the image viewer pop up, click “Actions” --> “Show unit cell tool”, which would pop up another window.



1. Lower the “Highest resolution for ring display” and hit enter to show the rings. In this case, 1.7 is a good value to display rings to the edge of the image.



2. If the red rings are a bit off with respect to the actual powder signals, you can explore some manual fitting here through “Detector Distance”, “Center fast” (x-axis, positive points to right), “Center slow” (y-axis, positive points down).

Notes and tips:

- The “Detector Distance” is auto-populated through the .expt file from psana geometry, but in practice we have seen some discrepancy between the values. Generally speaking, as long as the fit looks good, you can proceed and give indexing a try. See “Understand the detector distance number” section below for more details.
- If you manually refine the distance or the X/Y position in the “Unit cell tool” window, it won’t be saved to the .expt file. You would need to edit the values in the corresponding section in the .expt file. See “Understand the detector distance number” and “Modify beam X/Y (Center fast/slow)” sections below for more details.
- If you mis-enter something of the wrong format, the image viewer could just crash/freeze, you would have to start over.

Understand the detector distance number

⚠ As of 2025/11, there is some weird discrepancy between the value in the .expt file and the value populated in dials.image_viewer. We don’t fully understand why, but at least for mfx100904224 run 5 it seems okay enough to have some indexing results and refine geometry further.

Psana convention is to output detector distance as a negative number, and this is the convention followed by BayFAI. `dials.import` directly puts this negative number into the `.expt` file.

You can check this distance in the `{run_number}-end.data`, which is written by BayFAI to the `calib/{detectorname}/{somedetectorname}/geometry/` folder. An example file is: `/sdf/data/lcls/ds/mfx/mfx100904224/calib/Epix10ka2M: :CalibV1/MfxEndstation.0:Epix10ka2M.0/geometry/5-end.data`

Look at the last line that begins with "IP", in this case the distance would be `-106.405` (converted to mm).

```

# TITLE      Geometry parameters of ePix10k2M
# DATE_TIME  2025-08-04T13:40:07 PDT
# METROLOGY  no metrology available
# AUTHOR     lconreux
# EXPERIMENT N/A
# DETECTOR   Epix10ka2M
# CALIB_TYPE geometry
# COMMENT:01 Automatically created from BayFAI for the 16-segment ePix10k2M detector

# PARAM:01 PARENT      - name and version of the parent object
# PARAM:02 PARENT_IND  - index of the parent object
# PARAM:03 OBJECT      - name and version of the object
# PARAM:04 OBJECT_IND  - index of the new object
# PARAM:05 X0          - x-coordinate [um] of the object origin in the parent frame
# PARAM:06 Y0          - y-coordinate [um] of the object origin in the parent frame
# PARAM:07 Z0          - z-coordinate [um] of the object origin in the parent frame
# PARAM:08 ROT_Z       - object design rotation angle [deg] around Z axis of the parent frame
# PARAM:09 ROT_Y       - object design rotation angle [deg] around Y axis of the parent frame
# PARAM:10 ROT_X       - object design rotation angle [deg] around X axis of the parent frame
# PARAM:11 TILT_Z      - object tilt angle [deg] around Z axis of the parent frame
# PARAM:12 TILT_Y      - object tilt angle [deg] around Y axis of the parent frame
# PARAM:13 TILT_X      - object tilt angle [deg] around X axis of the parent frame

# HDR  PARENT  IND  OBJECT  IND  X0[um]  Y0[um]  Z0[um]  ROT-Z  ROT-Y  ROT-X  TILT-Z  TILT-Y  TILT-X
CAMERA 0 EPIX10KA:V1 0 -24363 -58909 247 0 0 0 -1.19820 -0.40996 0.02848
CAMERA 0 EPIX10KA:V1 1 -64974 -58190 431 0 0 0 -1.17006 -0.07098 -0.09996
CAMERA 0 EPIX10KA:V1 2 -23829 -17031 34 0 0 0 -0.88163 -0.24111 0.04677
CAMERA 0 EPIX10KA:V1 3 -64406 -16417 246 0 0 0 -1.12061 -0.19943 0.08481
CAMERA 0 EPIX10KA:V1 4 -59087 23990 109 270 0 0 -0.94903 0.42915 -0.00801
CAMERA 0 EPIX10KA:V1 5 -58412 64493 -8 270 0 0 -0.94784 0.41702 -0.00367
CAMERA 0 EPIX10KA:V1 6 -17273 23412 20 270 0 0 -0.81251 0.54338 -0.07746
CAMERA 0 EPIX10KA:V1 7 -16603 63851 -116 270 0 0 -0.89937 0.31273 0.33391
CAMERA 0 EPIX10KA:V1 8 23924 58637 -128 180 0 0 -1.18329 -0.26668 -0.28608
CAMERA 0 EPIX10KA:V1 9 64693 57890 -441 180 0 0 -1.09103 -0.38237 -0.09870
CAMERA 0 EPIX10KA:V1 10 23460 16641 -271 180 0 0 -0.91563 -0.17043 -0.34225
CAMERA 0 EPIX10KA:V1 11 63830 16051 -185 180 0 0 -0.92659 -0.17464 -0.25145
CAMERA 0 EPIX10KA:V1 12 58828 -24591 -97 90 0 0 -1.23985 0.16673 0.12936
CAMERA 0 EPIX10KA:V1 13 58090 -65326 34 90 0 0 -0.89964 0.20490 0.40665
CAMERA 0 EPIX10KA:V1 14 16663 -23885 -33 90 0 0 -0.66238 -0.11234 0.13809
CAMERA 0 EPIX10KA:V1 15 16061 -64217 154 90 0 0 -1.09612 0.19084 0.15380
IP 0 CAMERA 0 0 0 -106405 0 0 0 0.00000 0.00000 0.00000

```

And in the imported `.expt` file, the corresponding line about detector distance is:

```

2935 ],
2936 "hierarchy": {
2937   "name": "D0_R",
2938   "type": "",
2939   "fast_axis": [
2940     1.0,
2941     0.0,
2942     0.0
2943   ],
2944   "slow_axis": [
2945     0.0,
2946     1.0,
2947     0.0
2948   ],
2949   "origin": [
2950     0.0,
2951     0.0,
2952     -106.4050000000129
2953   ],
2954   "raw_image_offset": [

```

It would be the last number under “origin” under the first entry named “D0” or something similar immediately after “hierarchy”.

⚠️ Weird things we have observed as of 2025/11:

- The plus/minus sign matters. In `dials.image_viewer`, a positive number is always enforced. However, it appears that the handedness of coordinate system is preserved. If `bayFAI` says `-106.4`, but you manually remove the minus sign in `.expt` file, the image would flip upside down in the `dials.image_viewer`.
- There is some strange offset between `.expt` value and the value populated in the image viewer. For example, with `-106.4` for `mfx100904224` run 5, this is populated as 105.6 in the image viewer, but with `106.4` it is populated as 106.8.

Modify beam X/Y (Center fast/slow)

If you need to adjust the Center fast/slow entries in the “Unit cell tool” window of the image viewer, you need to save this result somehow by manually editing the `.expt` file. Suppose you need to adjust 50.0 and 40.0 for Center fast/slow.

Next you need to find the “`pixel_size`” in the `.expt` file. Do not look at the values near where you find the detector distance, as values would be 0 – instead, look for `pixel_size` near the top of the `.expt` file. In this case, it is `[0.1, 0.1]`. Then, multiply the values: this means that you would put down 5.0 and 4.0 instead of 50.0 and 40.0.

⚠️ The plus/minus sign, as mentioned above in the detector distance section, can be tricky. In the image viewer, you would find positive as going right and down, but because of the discrepancy between `psana` and `cctbx` coordinate system (as of 2025/11), positive in the `.expt` file means going left and down.

Putting together, this means that you would modify the “origin” in the screenshotted part of the .expt file from [0.0, 0.0, -106.405...] to [-5, 4, -106.405,...].

Refine the geometry with DIALS

Once you can successfully index some images, you can refine the geometry further using the indexed reflections. If needed, you can also refine the geometry again after running the ensemble refinement task (just update the file paths to point to ensemble refinement outputs rather than indexing outputs).

Use the first run in the run group we care about (e.g. run 21 for run group 58 trial 9 in this case).

0. Book-keeping:

go to the geom folder -- make one right under the experiment tag folder (e.g. common) if you haven't, e.g. /sdf/data/lcls/ds/mfx/mfx100904224/results/common/geom. Then, make a refine folder for the run group (e.g. refine_rg058) and go into this folder.

1. combine the indexing output files:

```
dials.combine_experiments /path/to/expt /path/to/refl [optional arguments]
```

For example, in the geom folder:

```
dials.combine_experiments ../../results/r0021/009_rg058/out/*refined*.  
expt ../../results/r0021/009_rg058/out/*indexed*.refl  
reference_from_experiment.detector=0 output.n_subset=1000  
output.n_subset_method=n_refl
```

2. Filter reflections:

```
cctbx.xfel.filter_experiments_by_rmsd combined.*
```

3. Make a 0-level refinement phil file:

or copy a previous one over:

```
cp  
/sdf/data/lcls/ds/mfx/mfx100903824/results/common/geom/refine_rg011/re  
fine_level0.phil ./
```

Example 0-level refinement phil file looks like this:

```
output.experiments=refined_level0.expt  
output.reflections=refined_level0.refl  
refinement {
```



```

parameterisation {
  auto_reduction {
    min_nref_per_parameter = 3
    action = fail fix *remove
  }
  beam {
    fix = *all in_spindle_plane out_spindle_plane wavelength
  }
}
refinery {
  engine = SimpleLBFGS LBFGScurvs GaussNewton LevMar *SparseLevMar
}
reflections {
  outlier {
    algorithm = null auto *mcd tukey sauter_poon
    separate_experiments = False
    separate_panels = True
  }
}
}

```

You might want to explore the outlier algorithms.

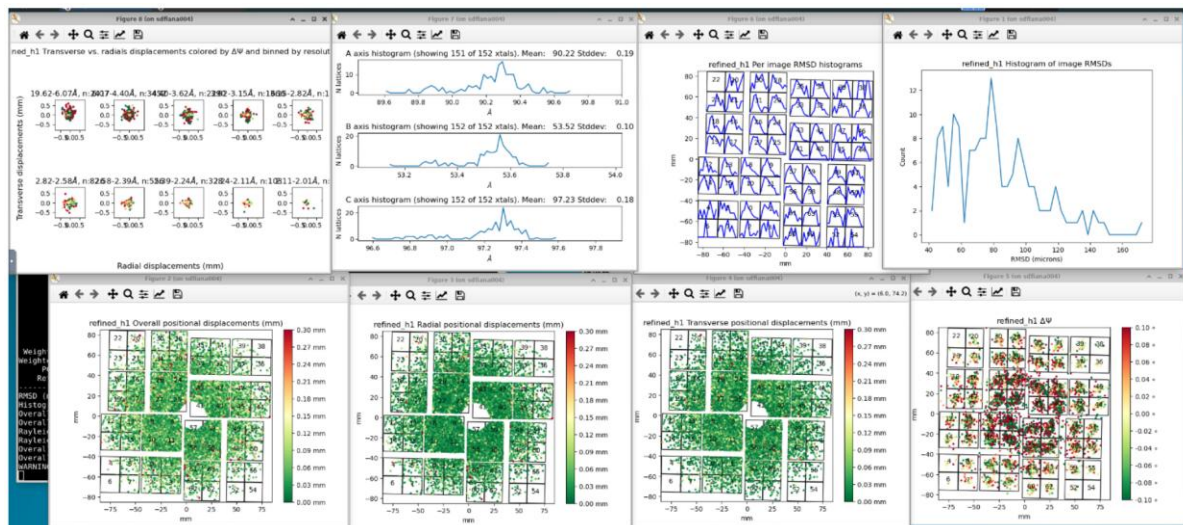
4. Perform 0-level refinement on the filtered reflections:

```
dials.refine filtered.* refine_level0.phil
```

5. Visualize the results and log the terminal output:

```
cctbx.xfel.detector_residuals refined_level0.* hierarchy=0 tag=refined
```

Example figures output:



Generally, we care more about the bottom row of figures, with the following rule of thumb:

- The greener plots the better
- The lower left three figures should have spots cover the detector well to the edge
- The “refined $\Delta\Psi$ ” figure should have spot concentrated into a small circular region in each panel center

Example terminal output:

```
RMSD (microns) 150.5531824354221
Histogram mode (microns): 132.32253458087138
Overall mean (microns): 133.50295828414986
Overall median (microns): 127.85311020060347
Rayleigh Mean (microns) 165.84170327562524
Rayleigh RMSD (microns) 187.13232301185118
Overall radial RMSD (microns) 125.05808894584378
Overall transverse RMSD (microns) 83.82401144692689
```

6. Perform 1-level refinement and check the visual results and RMSD improve.

Example 1-level refinement phil file:

```
output.experiments=refined_level1.expt
output.reflections=refined_level1.refl
refinement {
  parameterisation {
    auto_reduction {
      min_nref_per_parameter = 3
      action = fail fix *remove
    }
  }
  beam {
    fix = *all in_spindle_plane out_spindle_plane wavelength
  }
  detector {
    fix_list = Group1Tau1
    hierarchy_level = 1
  }
}
refinery {
  engine = SimpleLBFGS LBFGScurvs GaussNewton LevMar *SparseLevMar
}
reflections {
  outlier {
    algorithm = null auto *mcd tukey sauter_poon
    separate_experiments = False
    separate_panels = True
  }
}
```

```
}  
  }  
}
```

The command to visualize at level 1 is:

```
cctbx.xfel.detector_residuals refined_level1.* hierarchy=1  
tag=refined_h1
```

Intermediate: How to make a mask

This chapter assumes you have basic familiarity with `dials.image_viewer` and you know how to average a run. Refer to the previous chapter “Geometry Refinement” if you are not familiar with them.

Visualize a mask

Given a pre-made mask (`pixels.mask`), you can visualize it on an image (`std.cbf`):

```
dials.image_viewer std.cbf load_models=False mask=pixels.mask
show_mask=True
```

If you didn't specify `show_mask=True`, remember to check the box for “Show mask” in the “settings” panel that pops up, and the masked pixels will be displayed as red.

Make a mask

Steps 0-4 describe how to make masks. After averaging a run, go to the output folder in the terminal. In the following example, the folder is

`/sdf/data/lcls/ds/mfx/mfx101211025/results/doris/results/averages/r0016/000/out`, a case that requires extensive masking.

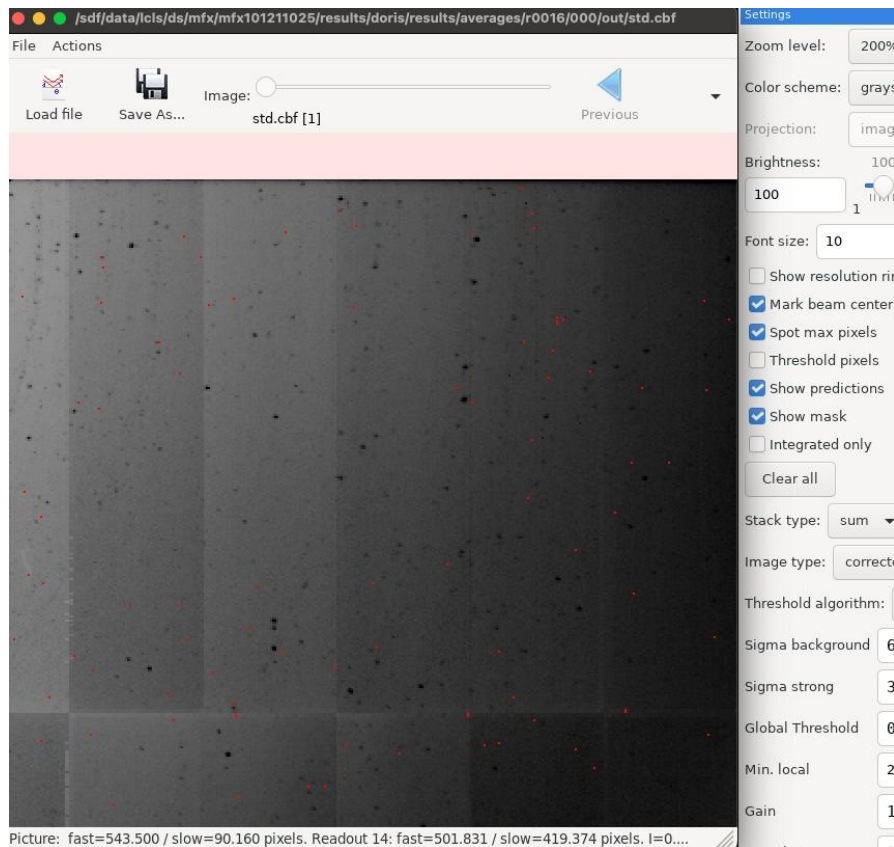
0. Dead pixels

If pixels have 0 standard deviation, they are probably dead pixels. To make a mask of them, use the following script from Fred:

```
(base) [mai12345@sdfiana023 000]$ libtbx.python
Python 3.12.11 | packaged by conda-forge | (main, Jun  4 2025,
14:45:31) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import dxtbx
>>> img = dxtbx.load("out/std.cbf") might be "max.cbf"
>>> mask = [m > 0 for m in img.get_raw_data()]
>>> from libtbx import easy_pickle
>>> easy_pickle.dump("r0016_stddev.mask", tuple(mask))
>>> exit()
```

Modify the file name or paths in the script as needed. If you are resuming from previous processing, make sure you have sourced the `cctbx` environment to get `libtbx.python`.

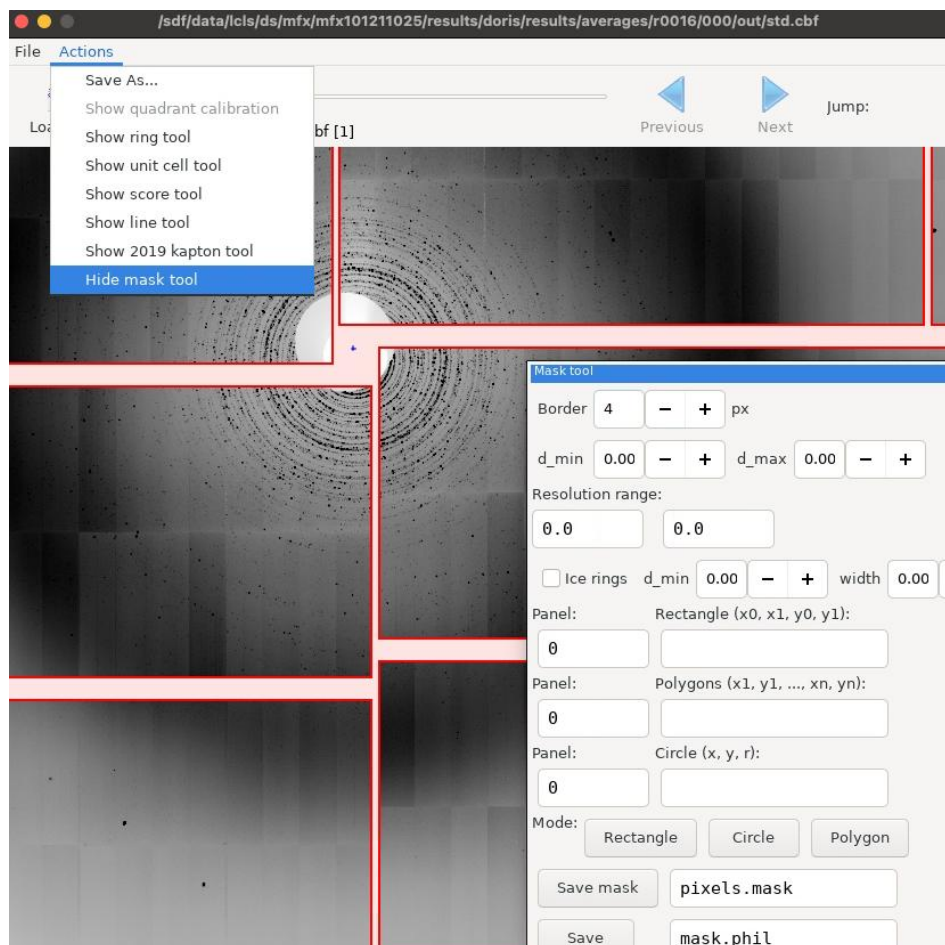
Dead pixels can be quite scattered like salt and pepper noise patterns.



1. Panel borders

Load the image with the dead pixel mask from the previous step as a starting point.

Click the “Mask tool” under the “Actions” button. To mask panel border, you can increase the value for the “border”.

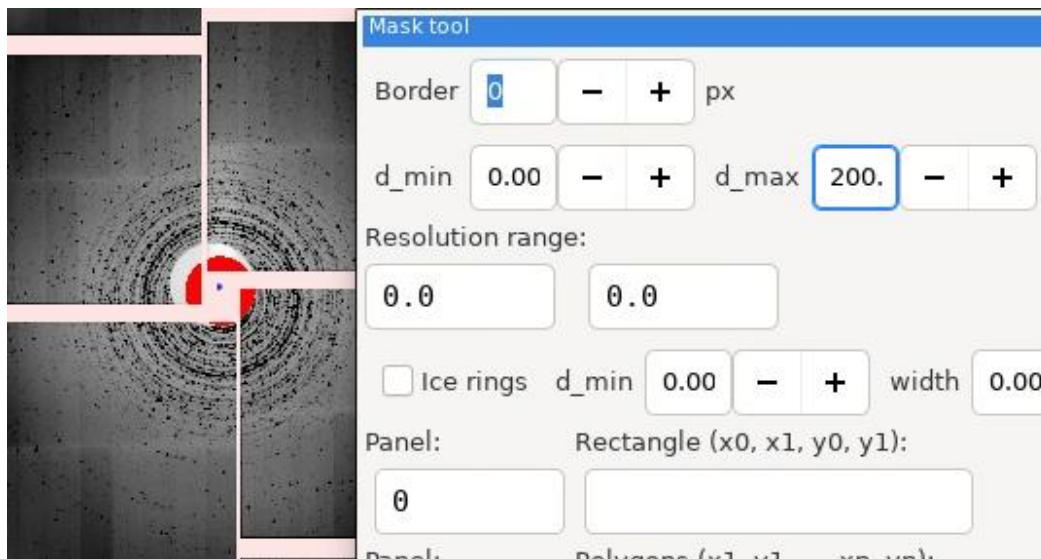


2. Beam center with a circle and mask by resolution range

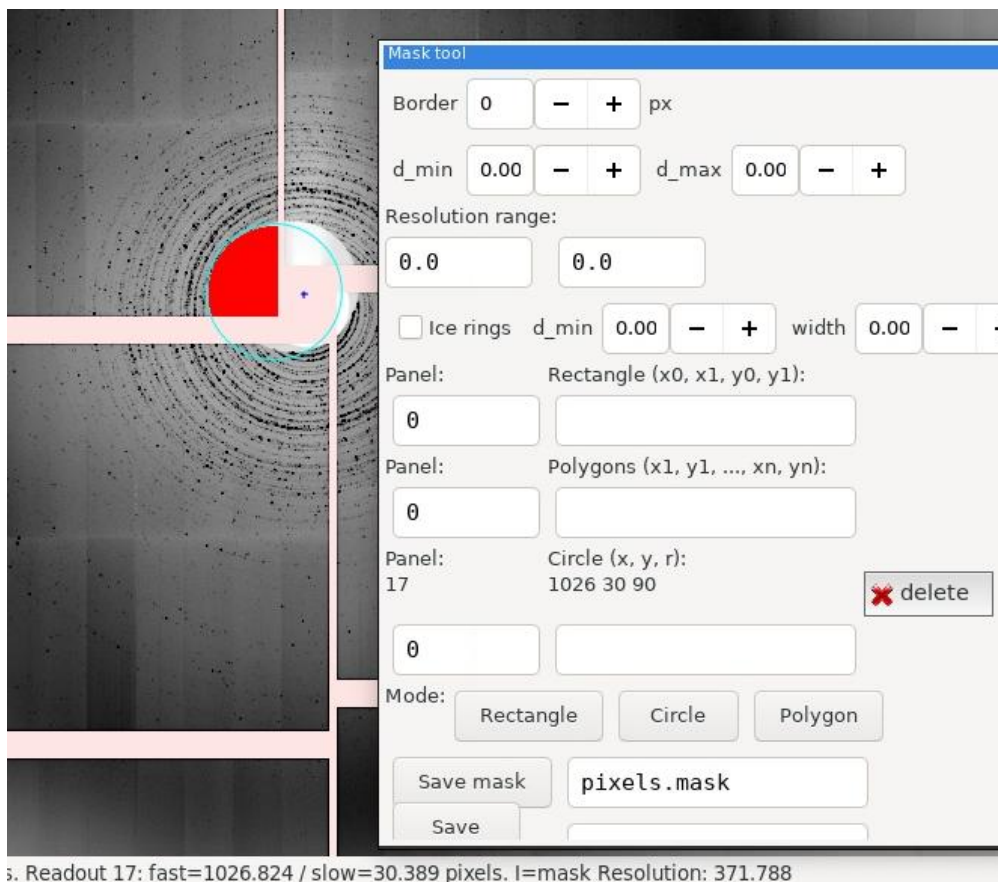
Open the Mask tool as shown in the previous step.

From my own exploration, it doesn't look like the Mask tool in dials.image_viewer GUI supports cross-panel geometry shape definition.

If the actual beam center is close to the what dials have (shown by the blue cross, default to image center I believe), you can hack by using resolution range masking. For example, make $d_{\max}$ 200.0:



However, as you can see in this case, the beam center is actually quite shifted. If you want to define an arbitrary circle, you will see that you have to define the circle center within a particular panel, and only the circle on that panel is masked:



In this case, I typed panel 17 and defined the circle (x,y,r) as 1026,30,90. I eyeballed the radius as 90 and got the panel and the position values by hovering my mouse over the

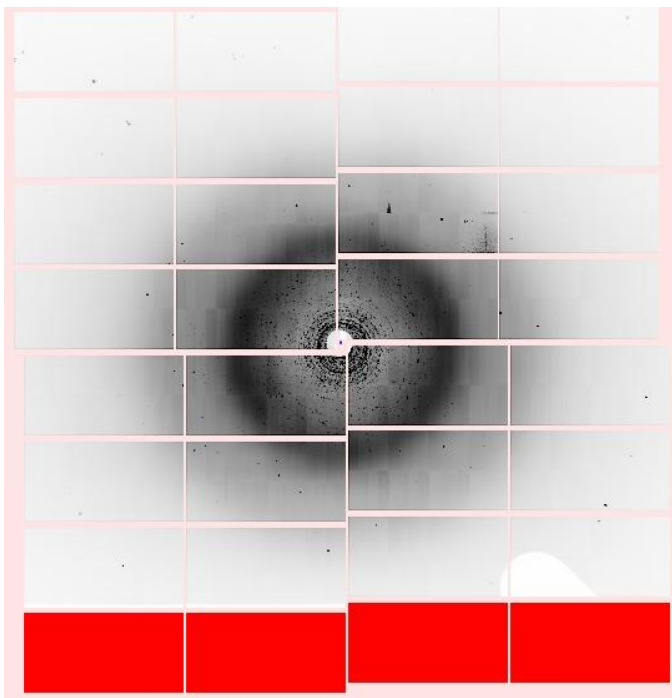
center, and at the bottom of the image you will see some text like “... Readout 17: fast=1026 / slow = 30 ...”.

3. Whole panel due to shadow

The easiest way to mask whole panels is actually using a command line tool. In this case, suppose I want to mask out the bottom panels which are panel 0-3 (judging from the “Readout” number as shown above). In the terminal:

```
dials.generate_mask out/std.cbf untrusted.panel=0 untrusted.panel=1  
untrusted.panel=2 untrusted.panel=3 output.mask=panel.mask
```

Visually verify if this mask looks good:



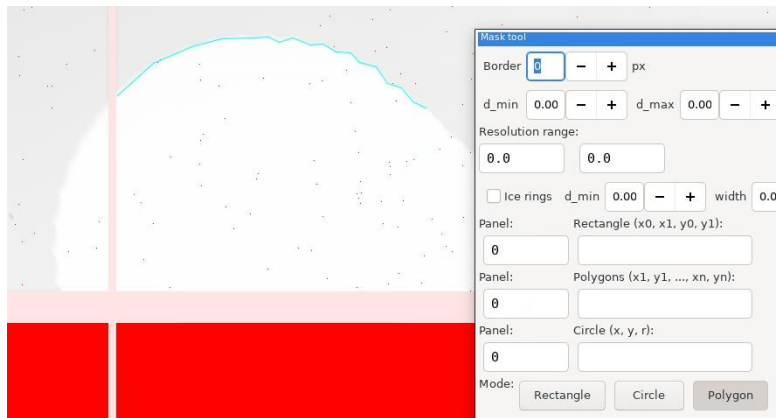
If the mask is good, combine this mask with your mask from other step (such as dead pixels) using the same tool:

```
dials.generate_mask r0016_stddev.mask panel.mask  
output.mask=combined.mask
```

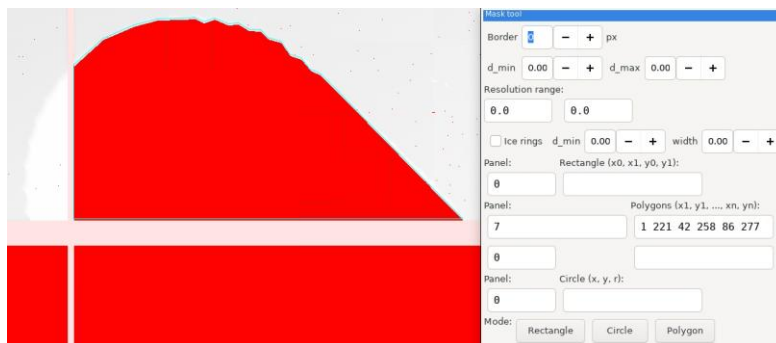
4. Irregular shape

The easiest way to make a polygon mask is by clicking where the vertices would be on an image. First open the Mask tool as shown step 1.

Click the Polygon button, and then if you click anywhere on the image, wherever you click would be the vertices. The order matters. Your trace / the edges of the polygon is shown as the cyan clines:



Once you roughly close the shape, click the Polygon button again, and that should properly close the polygon shape for you and mask the region:



5. Good practice notes

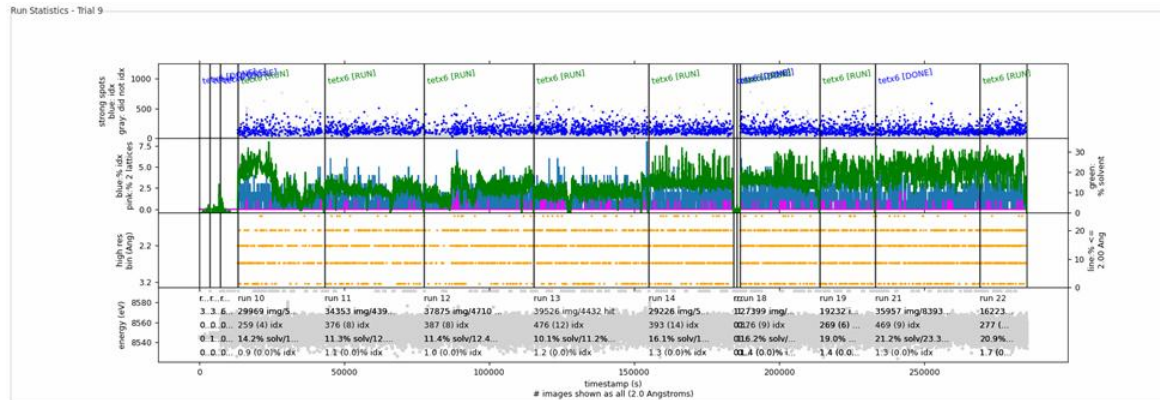
After you think you have made a decent mask by combining some of the steps above, you can save the final mask by clicking “Save mask”.

Below that button, you can also save the phil parameters of the masking you did through the image_viewer Mask tool, such as making a polygon mask. I strongly recommend you save this every time (change the file name to avoid overwriting) before you close the image_viewer (which you might need to open and close multiple times during a real processing). This is a quantitative way of recording you manual masking and helps reproduce.

Intermediate: Indexing result improvement

Diagnostics

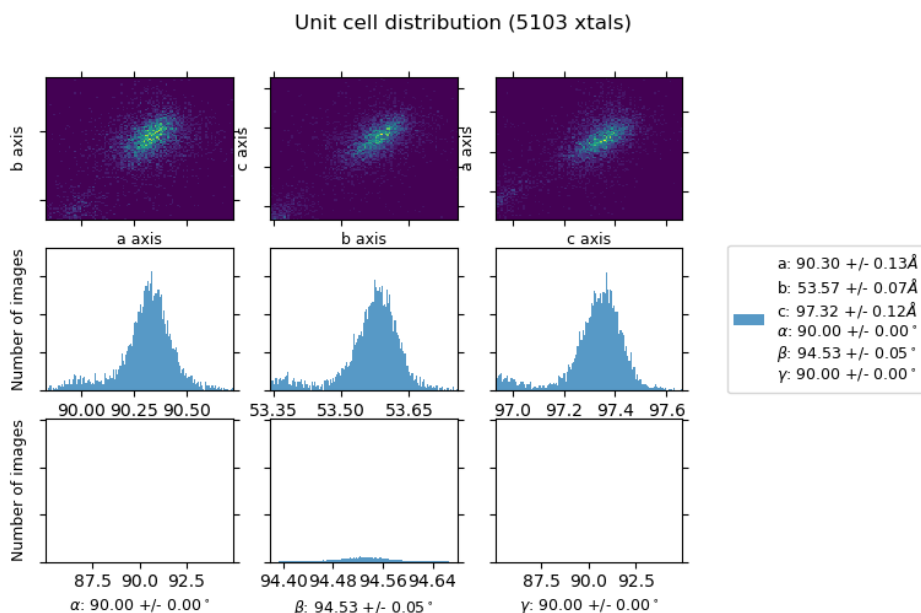
“Run Stats” tab of the GUI



You might need to hide Options to see the numbers at the bottom. "%xtal" (behind the %solv) is the hit rate and "%idx" is the indexing rate (also plotted as the blue line in the middle panel). The numbers in the parenthesis are number of images with multiple lattices.

I would also look at the top panel, where blue dot is indexed and grey dot is not, and ideally there are quite a bit of blue on top without too much grey stuff visible. It is possible to click on the dots to launch the corresponding image via `dials.image_viewer` to see if indexing and spotfinding works as expected.

“Unit Cell” tab of the GUI



Ideally we should see nice and narrow Gaussians centered at expected cell parameters.

Low indexing rate

Is your spot finding good? Are there clearly visible spots that are strong and lattice-like but not picked up somehow?

- Do you need to make a better mask?
- Explore via `dials.image_viewer` on the parameters for thresholding pixels

Once confirmed strong spots are found, explore indexing related knobs:

- Indexing algorithm parameters: consider using `indexing.stills.method_list=fft1d fft3d real_space_grid_search` to your phil parameter (trial definition or run group extra phil parameter).
- Geometry:
 - o Perform geometry refinement at both level 0 and 1. Could also explore different choices (mcd, tukey, sauter_poon) for `reflections.outlier.algorithm`
 - o Try proceed to the Ensemble refinement task for updated cell and geometry

Strange unit cell distribution

If the unit cell distribution shape is not a nice Gaussian (but skewed or bimodal):

- Do you need to (further) refine the geometry?
- Was there energy drift?

- Is there indexing ambiguity for this space group + unit cell?

Intermediate: Photon energy calibration

Correct energy can help indexing. To perform energy calibration, go to the “Energy” page of the GUI.

0. Populate the table

On the upper left panel, click “Add Row”, then click on each entry under “Run” and “Notch Energy” to manually fill a value. (You need to click “Add Row” for each new row to add.)

FEE Energy Scan Description

Experiment:

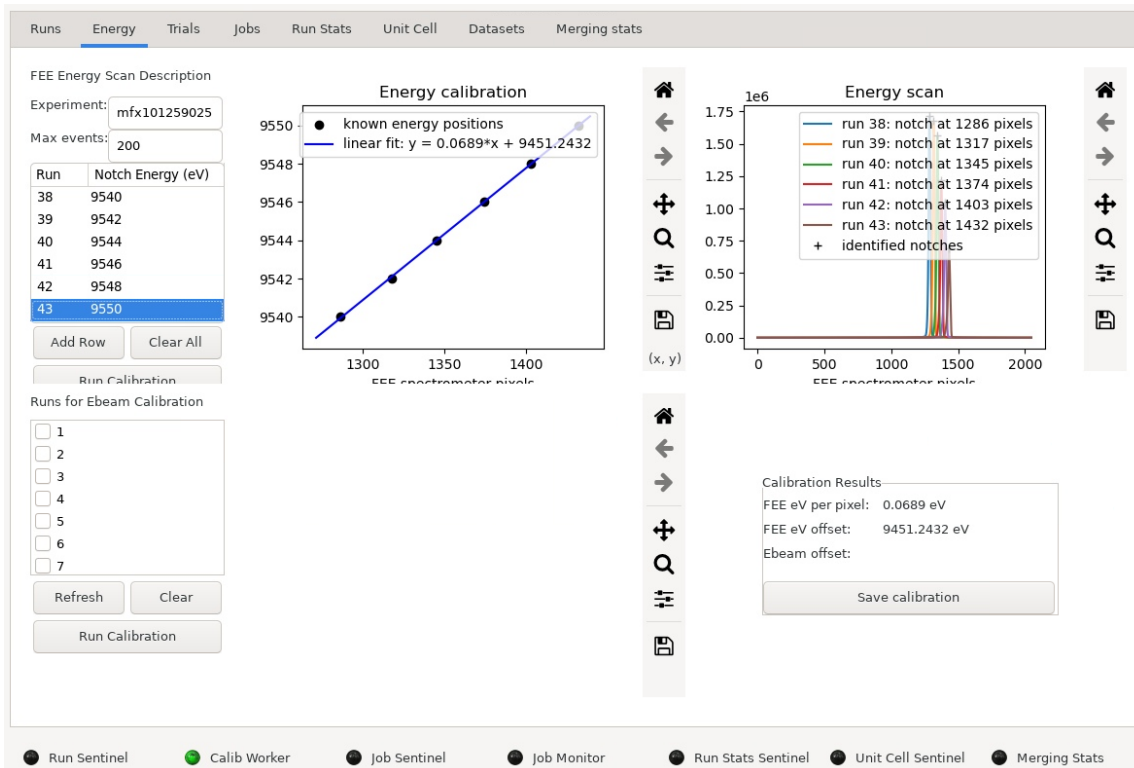
Max events:

Run	Notch Energy (eV)
38	9540
39	9542
40	9544
41	9546
42	9548
43	9550

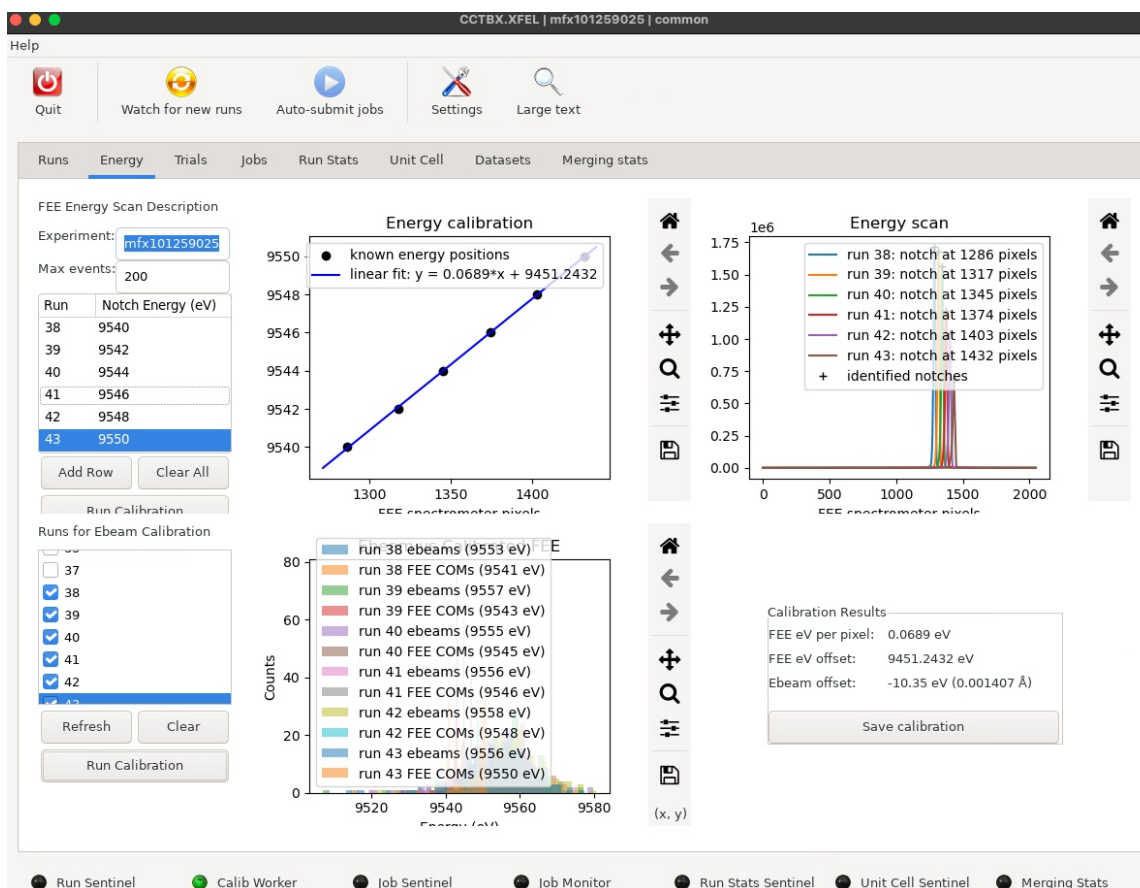
⚠ Only support one energy per run rather than a scan where many energies within a run.

1. Run Calibration

Once done, click “Run Calibration.” At the bottom, the sentinel for “Calib Worker” should turn yellow indicating work in progress. This would take a few seconds, and once done, the sentinel becomes green, and you should see something like this:



Select the same runs for Ebeam Calibration (lower left panel) and click “Run Calibration.” This can take a few minutes. Once it’s done, you should see another plot “Ebeam vs Calibrated FEE” populated and the value for “Ebeam offset” value should also be there under “Calibration results”:



⚠ If for some reason the job failed and the sentinel is red, clicking anything on this page might not get a response. After you make your corrections, you can try switch to a different page like “Trials” and then switch back to “Energy” to properly refresh the sentinel.

⚠ I am having trouble with other experiments for the “Ebeam Calibration.” For mfx100903824 (psana1), run 4-8 throws error on the terminal that these runs are not in any run group. For run 14, 16, and 18, I see the following error on the terminal: 'Unknown DetInfo device type: feespec (source: feespec)'. Unclear why and how to solve it.

⚠ I am having trouble with other experiments for the “Energy Calibration.” For mfx101262725 (psana2), run 225-234, I get the following error on the terminal: unsupported operand type(s) for !=: 'NoneType' and 'int'. Unclear why and how to solve it.

2. Use the results for run group definition

The calibration results are shown at the lower right panel, which you can save to a file if you click the button, and the file looks like this:

```
(base) [mai12345@sdfiana026 phil]$ cat fee_calib.phil
spectrum_eV_per_pixel=0.0689
```



```
spectrum_eV_offset=9451.2432  
ebeam_eV_offset=-10.35
```

Enter the two spectrum_eV related values when defining the run group (scroll down to see the corresponding entries in the GUI).

The screenshot shows a GUI with three main sections:

- Extra phil parameters:** Contains two buttons, "Import PHIL" and "Default PHIL", and a large text area currently displaying "None".
- Extra XTC format parameters:** Contains one button, "Import PHIL", and a large text area currently displaying "None".
- Options:** Contains several input fields:
 - Beam:** X (None), Y (None), DetZ (580.0)
 - binning:** (None), energy (None)
 - wavelength_offset:** (None)
 - spectrum_eV_per_pixel:** (None), **spectrum_eV_offset:** (None)

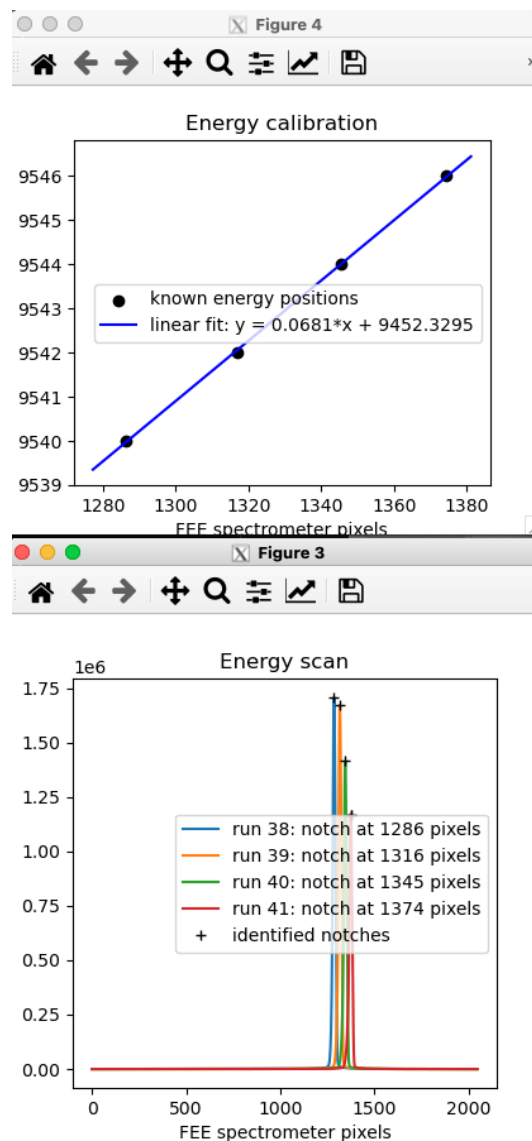
Alternative method

In case something goes wrong with the GUI, the corresponding command line tool for FEE calibration is `xfel.fee_calib <expname> <run>:<notch_energy>`. For example:

```
[mai12345@sdfiana003 ~]$ xfel.fee_calibration experiment=mfx101259025  
38:9540 39:9542 40:9544 41:9546  
Processing run 38...  
Found 1000 events with FEE, 0 events without (1000 total)  
Processing run 39...  
Found 2000 events with FEE, 0 events without (2000 total)  
Processing run 40...  
Found 3000 events with FEE, 0 events without (3000 total)  
Processing run 41...  
Found 4000 events with FEE, 0 events without (4000 total)
```

```
Calibrated eV offset of 9452.329454096232 and eV per pixel of
0.06813544632577721
wrote calibrated values to fee_calib.out
[mai12345@sdfiana003 ~]$ cat fee_calib.out
using experiment=mfx101259025 38:9540 39:9542 40:9544 41:9546,
eV_offset=9452.329454096232 eV_per_pixel=0.06813544632577721
```

The command should also pop out familiar figures like this:



Advanced: Event code handling

Helpful past experiments to look at include mfx1015222 – Lane and mfx101262725 – Follmer (and other T-jump experiments in the past). If there are issues that require addressing CCTBX source code, [dxtbx/src/dxtbx/format/FormatXTC.py](https://github.com/dxtbx/src/dxtbx/format/FormatXTC.py) is a key file to look at.

Sanity check if event code is set up properly for a run

The following script `list_event_codes.py` by Fred is helpful:

```
from psana import DataSource
from sys import argv

args = argv[1:]
exp = str(args[0])
run = int(args[1])
event_codes = [int(arg) for arg in args[2:]]

ds = DataSource(exp=exp, run=run, detectors=['timing'],
max_events=100)
myrun = next(ds.runs())
timing = myrun.Detector('timing')
for nevt, evt in enumerate(myrun.events()):
    allcodes = timing.raw.eventcodes(evt)
    evtcodes = [i for i, val in enumerate(allcodes) if val != 0]
    print(evtcodes)
```

To use this script, make sure you have activated psana2 environment, and then:

```
python list_event_codes.py <exp> <run>
```

To understand the output, for example, event code 201-213 (except for 208) was set up for this T-jump experiment, and the first few line of results look like this:

```
(ps_20241122) [mai12345@sdfiana002 dorismai]$ python list_event_codes.py mfx101262725
121
[0, 1, 9, 10, 40, 41, 42, 84, 88, 99, 101, 119, 122, 123, 128, 132, 135, 137, 140,
141, 142, 151, 152, 153, 155, 157, 158, 165, 201]
[0, 1, 9, 10, 11, 40, 84, 89, 99, 101, 119, 122, 123, 128, 132, 135, 137, 140, 151,
152, 153, 155, 157, 158, 202]
[0, 1, 9, 10, 40, 41, 84, 89, 99, 101, 119, 122, 123, 128, 132, 135, 137, 140, 141,
151, 152, 153, 155, 157, 158, 165, 203]
```

```
[0, 1, 9, 10, 11, 12, 13, 14, 15, 16, 40, 84, 88, 119, 122, 123, 128, 132, 135, 137, 140, 151, 152, 153, 155, 157, 158, 204]
[0, 1, 9, 10, 40, 41, 42, 43, 44, 45, 46, 84, 88, 99, 101, 119, 122, 123, 128, 131, 132, 135, 137, 140, 141, 142, 143, 144, 145, 146, 149, 151, 152, 154, 155, 157, 158, 159, 165, 169, 205]
[0, 1, 9, 10, 11, 40, 84, 88, 99, 101, 119, 122, 123, 128, 132, 135, 137, 140, 151, 152, 153, 155, 157, 158, 206]
```

You can see that the last number of each shot (each array) iterates among 201 to 213 as expected.

Specify event code in cctbx

You can filter to process data associated with only one event code by providing “Extra XTC format parameters” when defining a run group.

For example, for the T-jump experiment mfx101262725, event code 203 is reserved for the laser ON status. To process data associated laser on, you would put in the following lines:

```
filter.evr_address=evr0
filter.required_present_codes=203
```

These lines are added in addition to the other lines you might need, such as “mode=psana2_idx” for psana2 data.

Appendix: relevant programs and parameters

When tweaking PHIL parameters, it can be helpful to know what knobs are available. The command to do it is `[program name] -ca 2 -e 10`

Programs directly related to processing are:

`dials.stills_process` (for Indexing task)

`dials.combine_experiments` (for Ensemble Refinement)

`cctbx.xfel.merge` (for Scaling and Merging tasks)

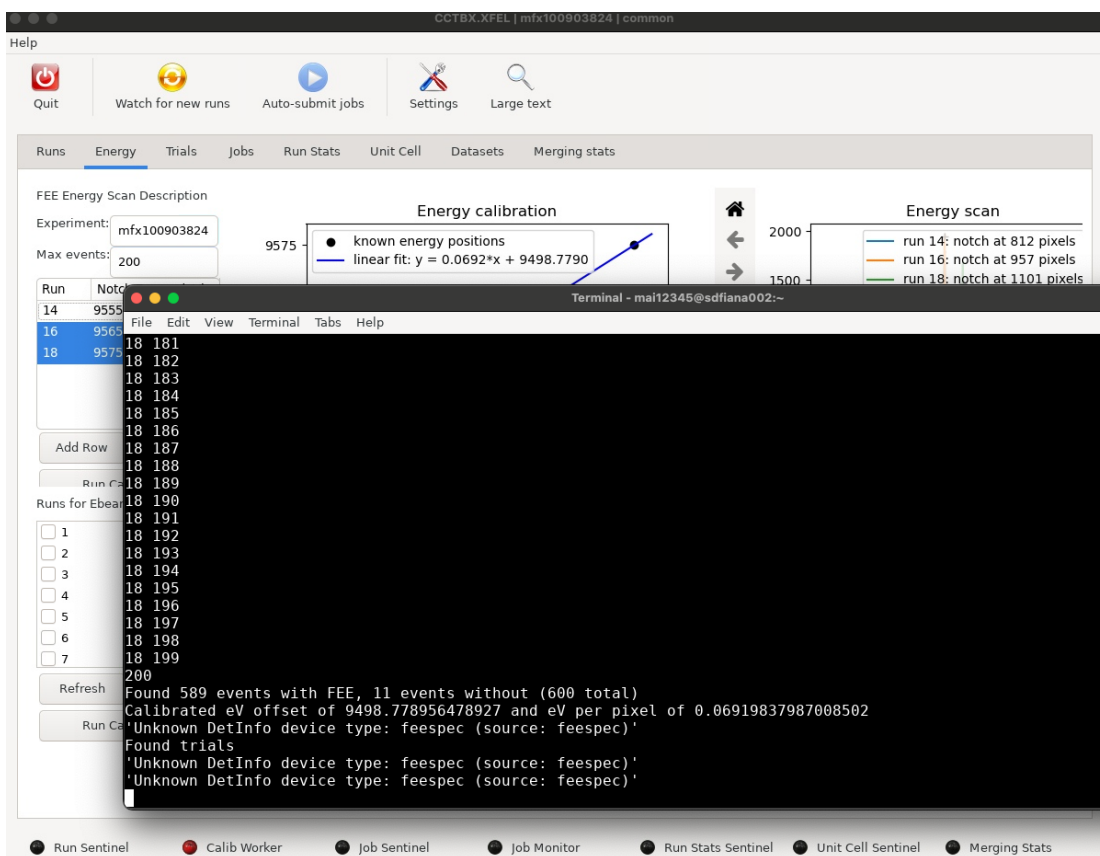
There are many other programs this handbook mentioned for diagnostics or other assisting purposes, such as `dials.image_viewer` and `cctbx.xfel.detector_residuals`.

Debugging tips

Review all the red warning texts listed previously in this document that begin with ⚠️, as the resulting errors are not necessarily covered here.

Failed/red sentinel / unresponsive GUI

Sometimes if a particular page of the GUI goes wrong (indicated by the red sentinel), it can become unresponsive. For example, in this case the ebeam calibration has errors and “Calib Worker” sentinel becomes red:



You might get away by switching to another page (e.g. “Trials”) and then come back to “Energy” page to re-run the calibration work.

No jobs submitted

Error:

Multiple lines on the terminal like:

Warning, expected to find submitted [xxx] job: trial [xxx], rungroupp[xxx], run [xxx]

Solution:

Check if you have properly defined the chain of tasks under the active dataset. In particular, did you have Indexing listed first?

If Indexing task was there, did it have `integrate=False` under the trial definition? If that's the case, you either need another indexing task that does integration or have ensemble refinement task here.

Look at parameters available to tweak for dials programs

`[dials program name] -ca 2 -e 10` is a good place to start.

Look at some example diffraction images

You can look through the raw images using `dials.image_viewer data.loc load_models=False`. And remember `data.loc` is just specifying parameters for psana, so you can make one from scratch / modify from another `data.loc` file rather than waiting for the GUI indexing task to generate for you.

0. Source the right cctbx environment. If necessary, maybe also check if that it was built in the right psana environment.
1. Make a `data.loc` file if you don't already have one. An example file with minimal entries look like this:

```
experiment=mx101262725
run=532
detector_address=jungfrau
mode=psana2_idx # only for psana2
```

This is only a minimal set of entries. You might need other things like specifying wavelength fallback and spectrum address etc. depending on the errors you get when launching `dials.image_viewer`. Refer to “Check FormatXTC and `data.loc`” section below (and maybe also “Check psana detnames output” section).

2. Launch images via the terminal:

```
dials.image_viewer data.loc load_models=False
```

This command lets you see the unindexed images of a run.

To look at indexed images, you would replace `data.loc` with the paths to the corresponding `.expt` and `.refl` files in the command. For example:

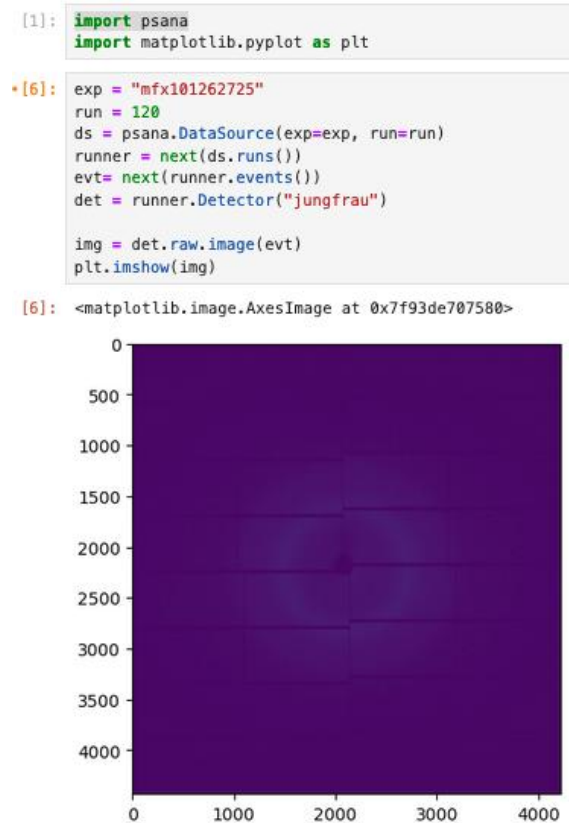
```
dials.image_viewer out/idx-050_*. {expt,refl} load_models=False
```

If you encounter errors with `dials.image_viewer`, refer to the “Dials.image_viewer related issues” section below.

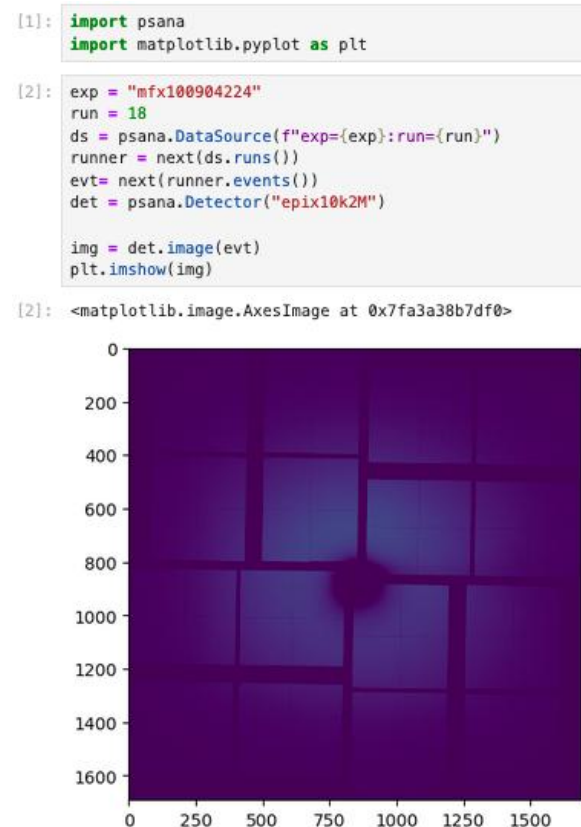
Visualize directly through psana in Jupyter Notebook

Not recommended, as it is very inefficient, but if you must do this for some reason, open a jupyter notebook through ondemand interactive session, and use the following code snippets depending on psana version.

Psana2 (left)



Psana1 (right)



Check psana detnames output

This can be useful to check if entries are missing (e.g. eBeam), and correct names of detector/spectrum addresses to access XTC files via data.loc (either manually added or provided as “Extra xtc format parameters” in the GUI) or custom psana code.

For psana1: detnames exp=<exp>:run=<run>

For psana2: detnames exp=<exp>,run=<run>

Psana2 have additional flags to enable looking at scan data for example.

Check FormatXTC and data.loc

Failure of this can manifest as failed to submit indexing jobs, as well as failure to open `dials.image_viewer`. FormatXTC is a script in `cctbx`:
`dxtbx/src/dxtbx/format/FormatXTC.py`.

All the solutions below that involve manually editing `data.loc` file are just for debugging purposes. To properly generate the `data.loc` file, remember to enter the lines into `cctbx.xfel` GUI run group definition as “Extra xtc format parameters”.

Spectrum/Detector address – 2025/11:

Error:

Traceback (most recent call last): File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/build/./modules/dials/src/dials/command_line/image_viewer.py"`, line 213, in `run()` File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/conda_base_psana1/lib/python3.11/contextlib.py"`, line 81, in inner return `func(*args, **kwargs)` ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/build/./modules/dials/src/dials/command_line/image_viewer.py"`, line 209, in `run show_image_viewer(params=params, reflections=reflections, experiments=experiments)` File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/build/./modules/dials/src/dials/command_line/image_viewer.py"`, line 174, in `show_image_viewer`

`wrapper.display(experiments=experiments, reflections=reflections)` File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/spotfinder_wrap.py"`, line 86, in `display self.frame.load_image(chooser_wrapper(imagesets[0], 0))` File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/spotfinder_frame.py"`, line 657, in `load_image super().load_image(` File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/slip_viewer/frame.py"`, line 438, in `load_image self.pyslip.tiles.set_image(` File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/slip_viewer/tile_generation.py"`, line 70, in `set_image`

`self.raw_image.set_image_data(get_image_data(self.raw_image))`

^^ File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/spotfinder_frame.py"`, line 851, in `get_image_data image_data = image.get_image_data()`

^^ File

`"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/slip_vi`

```

ewer/frame.py", line 57, in get_image_data return
self.image_set.get_corrected_data(self.index)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/imageset.py", line 290,
in get_corrected_data return super().get_corrected_data(index)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatMultima
ge.py", line 47, in read return format_instance.get_raw_data(index)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTCEpix.
py", line 38, in get_raw_data d = FormatXTCEpix.get_detector(self, index)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTCEpix.
py", line 65, in get_detector return FormatXTCEpix._detector(self, index)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTCEpix.
py", line 93, in _detector wavelength = self.get_beam(index).get_wavelength()
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTC.py",
line 566, in get_beam return self._beam(index) ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTC.py",
line 593, in _beam spectrum = self.get_spectrum(index) ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTC.py",
line 637, in get_spectrum spectrum = self._spectrum(index) ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTC.py",
line 657, in _spectrum self._fee = psana.Detector(self.params.spectrum_address)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/conda_base_psana1/lib/python3.11/site-
packages/psana/det_interface.py", line 58, in Detector return _detector_factory(name,
env, accept_missing=accept_missing)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/conda_base_psana1/lib/python3.11/site-
packages/Detector/PyDetector.py", line 111, in detector_factory dtype =
dettype(source_string, env, *args, **kwargs)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/conda_base_psana1/lib/python3.11/site-
packages/Detector/PyDetector.py", line 228, in dettype raise KeyError('Unknown DetInfo
device type: %s (source: %s)'

```

KeyError: 'Please report this error at <https://github.com/dials/dials/issues> or to dials-user-group@jiscmail.ac.uk: Unknown DetInfo device type: feespec (source: feespec)'

Solution:

First check psana output validity:

For example, in this psana1 experiment:

```
(ana-4.0.66-py3) [fpoitevi@sdfiana004 009_rg027]$ detnames  
exp=mx100904224:run=8
```

Full Name	DAQ Alias	User Alias
FEE-SPEC0		
MFX-DG1-BMMON		
MFX-DG2-BMMON		
NoDetector.0:Evr.0	evr0	
MfxEndstation.0:Epix10ka2M.0	epix10k2M	
ControlData		

This means in the data.loc file, add a line:

```
spectrum_address=FEE-SPEC0
```

Missing eBeam – 2025/11

Error:

Traceback (most recent call last): File

"/sdf/group/lcls/ds/tools/cctbx/psana1/build/./modules/dials/src/dials/command_line/image_viewer.py", line 213, in run() File

"/sdf/group/lcls/ds/tools/cctbx/psana1/conda_base_psana1/lib/python3.11/contextlib.py", line 81, in inner return func(*args, **kwargs) ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File

"/sdf/group/lcls/ds/tools/cctbx/psana1/build/./modules/dials/src/dials/command_line/image_viewer.py", line 209, in run show_image_viewer(params=params, reflections=reflections, experiments=experiments) File

"/sdf/group/lcls/ds/tools/cctbx/psana1/build/./modules/dials/src/dials/command_line/image_viewer.py", line 174, in show_image_viewer

wrapper.display(experiments=experiments, reflections=reflections) File

"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/spotfinder_wrap.py", line 86, in display self.frame.load_image(chooser_wrapper(imagesets[0], 0)) File

"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/spotfinder_frame.py", line 657, in load_image super().load_image(File

```

"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/slip_viewer/frame.py", line 417, in load_image img = rv_image(file_name_or_data)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/cctbx_project/rstbx/viewer/init.py", line
248, in init detector = self._raw.get_detector() ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/image_viewer/slip_viewer/frame.py", line 43, in get_detector return self.image_set.get_detector()
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/imageset.py", line 242,
in get_detector return self._get_item_from_parent_or_format("detector", index)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/imageset.py", line 236,
in _get_item_from_parent_or_format item = getter_function(self.indices())[index]
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTCEpix.py", line 65, in get_detector return FormatXTCEpix._detector(self, index)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTCEpix.py", line 93, in _detector wavelength = self.get_beam(index).get_wavelength()
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTC.py",
line 566, in get_beam return self._beam(index) ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dxtbx/src/dxtbx/format/FormatXTC.py",
line 627, in _beam if self._beam_cache.get_wavelength() < 0.1:
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

AttributeError: Please report this error at <https://github.com/dials/dials/issues> or to dials-user-group@jiscmail.ac.uk: 'NoneType' object has no attribute 'get_wavelength'

Solution:

In the data.loc file, add a line like:

```
wavelength_fallback=1.443
```

'NoneType' object has no attribute 'mode' – 2025/11

Error:

Sorry: Unable to handle the following arguments: "idx-0008_refined.expt" failed during ExperimentList processing: Traceback (most recent call last): File

```
"/sdf/group/lcls/ds/tools/cctbx/psana1/modules/dials/src/dials/util/options.py", line 321,
```

[illegible]

Solution:

This is very non-descriptive error message, which is NOT referring to the mode=idx or mode=psana2_idx. Instead, I think this error shows up when you have a typo in the data.loc file. For example, it shows when I misspelled wavelength_fallback.

Dials.image_viewer related issues – 2025/11

If you encounter errors when trying to visualize images through dials.image_viewer, explore the debugging approaches below.

- Check psana detnames output (see section above)
- Check FormatXTC and data.loc (see section above)

Check psana versions in cctbx build:

In the terminal: `libtbx.python -c "import psana; print(getattr(psana, 'xtc_version', None))"`

If psana1, should return None.

In the terminal: `libtbx.python -c "from dxtbx.format.FormatXTCEpix import FormatXTCEpix; FormatXTCEpix('data.loc')"`

With psana1, should return no error.

Check psana geometry

This can be an issue for psana2 for example, when BayFAI was deployed but the result was not successfully pushed to the database. To quickly check if psana understand the geometry correctly (is it default metrology or BayFAI) for GUI to access, you can use the following code snippets provided by Louis to check.

Psana2

```
exp = "mfx101262725"
run = 130
ds = psana.DataSource(exp=exp, run=run)
runner = next(ds.runs())
evt = next(runner.events())
det = runner.Detector("jungfrau")
```

📄 ↑ ↓

Psana1

```
exp = "mfx100904224"
run = 18
ds = psana.DataSource(f"exp={exp}:run={run}")
runner = next(ds.runs())
evt = next(runner.events())
det = psana.Detector("epix10k2M")
```

Plot image

+ 1 cell hidden

Detector distance

```
#psana2
geo = det.raw._det_geo()
x, y, z = geo.get_pixel_coords()
z.mean()
```

-102458.87499999999

```
#psana1
from PSCalib.GeometryAccess import GeometryAccess
geo = GeometryAccess(f"/sdf/data/lcls/ds/mfx/{exp}/calib/Epix10ka2M::CalibV1/MfxEndstation.0:Epix10ka2M.0/geometry/5-end.data")
x, y, z = geo.get_pixel_coords()
z.mean()
```

-106405.25

Database related – 2025/11

If you suspect issues related to the database, e.g. mysql error shows up in the terminal as you launch the GUI or attempt to submit jobs through the GUI, you can do some basic checking with mysql command in the terminal:

```
mysql -h 172.24.5.182 -u <experiment> <experiment> -p
```

After typing password, some helpful commands to explore include (semicolon is needed):

```
mysql> show tables;
```


Tables_in_mfxl1008021
common_beam
common_bin
common_cell
common_cell_bin
common_crystal
common_dataset
common_dataset_tag
common_dataset_task
common_dataset_version
common_dataset_version_job
common_detector
common_event
common_experiment
common_imageset
common_imageset_event
common_isoform
common_job
common_run

```
SELECT * FROM common_job LIMIT 10;
```

```
SELECT * FROM common_task LIMIT 3;
```

Failed jobs, hanging job, and resubmit jobs – 2025/10

You might want to resubmit jobs that failed or exited but you think they should have worked (e.g. failed due to xtc file transfer not complete yet). I have also seen at reprocessing stage, a few jobs just mysteriously fail the first time submitted but second time runs fine.

Alternatively, you might know something went wrong but the jobs haven't finished or appear hanging forever. Typically, a cctbx job should not run more than 10 minutes. In those cases, you want to manually kill the jobs.

After you see the status of a job becomes "EXIT", you can click on that job row (or select across multiple rows) and then click "Restart job" at the bottom right. It might take a few seconds for the old job to disappear and a new job to appear (remember to have the "Auto-submit jobs" turned on).

CCTBX.XFEL | mfxl1008021 | doris

Help

Quit

Watch for new runs

Auto-submit jobs

Settings

Large text

Runs

Energy

Trials

Jobs

Run Stats

Unit Cell

Datasets

Merging stats

Job	Type	Dataset	Trial	Run	Block	Task	Subm ID	Status	
266	-	-	t000	r0046	rg005	-	15843763	EXIT	
263	-	-	t000	r0045	rg005	-	15843762	EXIT	
260	-	-	t000	r0044	rg005	-	15843761	EXIT	
257	-	-	t000	r0043	rg005	-	15843760	EXIT	
254	-	-	t000	r0042	rg002	-	15843759	EXIT	
251	-	-	t000	r0041	rg002	-	15843758	EXIT	
248	-	-	t000	r0040	rg002	-	15843757	EXIT	
245	-	-	t000	r0039	rg002	-	15843756	EXIT	
242	-	-	t000	r0038	rg002	-	15843755	EXIT	
239	-	-	t000	r0037	rg002	-	15843754	EXIT	
236	-	-	t000	r0036	rg002	-	15843753	EXIT	
233	-	-	t000	r0035	rg002	-	15843752	EXIT	
230	-	-	t000	r0034	rg002	-	15843751	RUN	
227	-	-	t000	r0033	rg002	-	15843750	EXIT	
224	-	-	t000	r0032	rg002	-	15843749	EXIT	
221	-	-	t000	r0031	rg002	-	15843748	EXIT	
218	-	-	t000	r0030	rg002	-	15843747	EXIT	
215	-	-	t000	r0029	rg002	-	15843746	EXIT	
212	-	-	t000	r0028	rg002	-	15843745	EXIT	
209	-	-	t000	r0027	rg002	-	15843744	EXIT	
206	-	-	t000	r0026	rg002	-	15843743	EXIT	
203	-	-	t000	r0025	rg002	-	15843742	EXIT	
200	-	-	t000	r0024	rg002	-	15843741	EXIT	
197	-	-	t000	r0023	rg002	-	15843740	EXIT	
194	-	-	t000	r0022	rg002	-	15843739	EXIT	

Filter by:

All jobs

☒ Only display jobs from active trials/blocks

Stop job

Delete job

Restart job

To avoid hanging jobs due to premature start before xtc file transfer finishes, during beamtime (real-time) processing, I might sometimes disable and then re-enable “Autosubmit jobs” to create a little delay to avoid that situation.

Failure to index – 2025/11

This section assumes that the indexing job cannot be submitted or cannot properly finish. If the indexing job apparently finishes without concerning errors, look at the section below “Low indexing rate” -- in particular, is your initial geometry totally off?

- Are the XTC files there? This is important to check if you are reprocessing old experiments which might have data moved to tape. If there are no .xtc files directly under /sdf/data/lcls/ds/mfx/experiment_number/xtc, you need to contact

Wilko/pcds-datamgt-l email list. Once the files transfer is done, check if it has the correct permission (read for ps-data group).

- Is your data.loc file correct? Is cctbx build in the right psana environment? Refer to “Check psana detnames output” and “Check FormatXTC and data.loc” sections above. A quick test is if you can successfully launch `dials.image_viewer` to look at some images using the GUI generated data.loc.
- Are you missing ebeam and need wavelength_fallback information added to data.loc? Refer to “Check if eBeam is in xtc” section below and “Check FormatXTC and data.loc” section above.

Low indexing rate – 2025/11

- Is it a bad/short run? Do you have any image that looks like good diffraction? Identifying a few good ones is helpful for testing and improving indexing related parameters. Refer to the section “Look at some example diffraction images” above.
- Do you at least have good spot finding (hit finding) rate? If not, explore those parameters first in `dials.image_viewer`. Refer to the chapter “Spot finding parameters exploration.”
- Do you have proper masking of pixels that shouldn’t be identified as spot? Refer to the chapter “How to make a mask”.
- If you have okay hit rate but low indexing rate, have you refined the geometry? Refer to the chapter “Geometry Refinement”

Check if eBeam is in xtc– 2025/11

There is a helpful script from Fred (only for psana2). The command to use it is:

```
python test_ebeam.py --exp <experiment> --run <run>
```

The full script is attached below:

```
from psana import DataSource
import argparse
parser = argparse.ArgumentParser(description="Get a list of ebeam
photon energies from psana2. Make sure you are in psana2
environment.")
parser.add_argument('--exp', required=True, type=str, help='Experiment
name.')
parser.add_argument('--run', required=True, type=int, help='Run
number.')
parser.add_argument('--nevents', required=False, type=int,
default=1000, help='Number of events to process. Defaults to 1000.')
```

```

args = parser.parse_args()

exp = args.exp
run = args.run
nevents = args.nevents

ds = DataSource(exp=exp, run=run, detectors=['ebeamh'],
max_events=nevents)
myrun = next(ds.runs())
nevent=0
damaged=0
for nevt,evt in enumerate(myrun.events()):
    nevent+=1
    ebeam = evt.run().Detector('ebeamh')
    hu = ebeam.raw.ebeamPhotonEnergy(evt)
    if hu is None:
        damaged+=1
    else:
        print(hu)
print(f'{damaged}/{nevent}')

```

Check if event codes are in xtc – 2025/10

Similar to the script above to check ebeam, for the core piece of code, replace with follows:

```

ds = DataSource(exp=exp, run=run, max_events=nevents)
myrun = next(ds.runs())
timing = myrun.Detector('timing')
for nevt,evt in enumerate(myrun.events()):
    allcodes = timing.raw.eventcodes(evt)
    # make a list of indexes where allcodes is 1
    usedcodes = [i for i, code in enumerate(allcodes) if code == 1]
    print(usedcodes)
    if nevt >= nevents: break

```

Failure to merge

If merging job appears “EXIT”, is it because of no google drive publishing parameters provided (which is harmless, the job actually finished fine), or something else? Check the err.out to confirm.

Workaround (last resort) if previous processing went very wrong

If something previous tasks went really wrong that prevented you from continue (e.g. submitting new tasks for new datasets), a last resort is to make a new experiment tag and start over (all the way from tagging the runs). This happened to mfxl1008021, where at some point, a custom build was used to run abismal task. In later attempts to reprocess using the official build, abismal task is not supported and considered NoneType in the processing database that prevented submitting any new task.